ifotools

Andreas Freise, Miron van der Kolk, Timo Marchand, Yashwant Bothra

October 15, 2026

Contents

1 Introduction **1**

2 Finesse model tools: `ifotools.finesse` **3**

2.1 Model information: `finesse.model_info` . . . 3

2.2 Plotting: `finesse.plotting` . . . 9

2.3 Pretuning: `finesse.pretuning` . . . 14

2.4 Etalons: `finesse.etalon` . . . 17

2.5 Status types: `finesse.status` . . . 19

2.6 Utilities: `finesse.utils` . . . 21

3 Virgo tools: `ifotools.virgo` **25**

3.1 Model information: `virgo.model_info` . . . 25

3.2 Plotting: `virgo.plotting` . . . 28

3.3 Pretuning: `virgo.pretuning` . . . 29

3.4 Pretuning code generator: `virgo.pretuning_generator` . . . 31

4 Mirror maps: `ifotools.mirror_maps` **33**

4.1 The ET mirror data container: `mirror_maps.container` . . . 33

4.2 Map utilities: `mirror_maps.utils` . . . 42

4.3 Zernike polynomials: `mirror_maps.zernike` . . . 68

4.4 Hermite-Gauss modes: `mirror_maps.hermite_gauss` . . . 85

4.5 Power spectral densities: `mirror_maps.psd` . . . 90

4.6 Optical coupling: `mirror_maps.optical` . . . 101

4.7 Colour map: `mirror_maps.seasons_s` . . . 104

5 Beam propagation and gratings **105**

5.1 FFT propagation: `fft_propagation` . . . 105

5.2 Gratings: `grating` . . . 110

6 General utilities **113**

6.1 Top-level functions: `ifotools` . . . 113

6.2 Searching the package: `helper` . . . 113

6.3 SymPy helpers: `sympy_helpers` . . . 115

6.4 General utilities: `utils` . . . 116

6.5 Version: `version` . . . 118

Index of functions and classes **119**

CHAPTER 1

Introduction

Welcome to ifotools documentation. Below you will find a reference for the ifotools functions. Examples notebooks can be found at the gitlab repo. The package includes of submodules, and subpackages as shown in the table 1.1.

Module	Content	Section
Subpackage `ifotools.finesse` — tools for Finesse models		
`finesse.model_info`	Tables of powers, DOFs, locks, sensing matrices, tunings; model comparison	2.1
`finesse.plotting`	DOF scans, error signals, compass plots, sensing matrix plots	2.2
`finesse.pretuning`	Adjusting cavity and path lengths, demodulation phases	2.3
`finesse.etalon`	Detection of etalons formed by HR/AR surfaces	2.4
`finesse.status`	Status enumerations used in the model summaries	2.5
`finesse.utils`	Model loading, element filtering, scan specifications	2.6
Subpackage `ifotools.virgo` — tools for the Virgo model		
`virgo.model_info`	Model state, powers, quantum noise, lengths and frequencies	3.1
`virgo.plotting`	Quantum noise limited sensitivity plots	3.2
`virgo.pretuning`	Recycling cavity lengths and pretuning	3.3
`virgo.pretuning_generator`	Generator for pretuning code	3.4
Subpackage `ifotools.mirror_maps` — mirror surface maps		
`mirror_maps.container`	The ET mirror data container (HDF5)	4.1
`mirror_maps.utils`	Loading, saving, plotting and preparing maps	4.2
`mirror_maps.zernike`	Zernike polynomial analysis and synthesis	4.3
`mirror_maps.hermite_gauss`	Hermite-Gauss mode decomposition, coupling and loss	4.4
`mirror_maps.psd`	Power spectral densities and virtual maps	4.5
`mirror_maps.optical`	Optical coupling with Finesse	4.6
`mirror_maps.seasons_s`	Colour map used by `plot_map`	4.7
Submodules		
`fft_propagation`	FFT beam propagation	5.1
`grating`	Grating surface generation	5.2
`helper`	Searching the package	6.2
`sympy_helpers`	Printing SymPy expressions	6.3
`utils`	General utilities	6.4
`version`	Package version	6.5

Table 1.1: Modules of the `ifotools` package.

CHAPTER 2

Finesse model tools: `ifotools.finesse`

2.1 Model information: `finesse.model_info`

Function	Purpose
`compare_models`	Table of the parameter differences between models
`get_phase_config_status`	Phase configuration state of a model
`get_powers`	Carrier and total power at selected locations
`print_powers`	Table of the carrier and total power at selected locations
`print_dc_readout_powers`	Table of the powers at the DC readouts
`print_dofs`	Table of the degrees of freedom
`print_lock_status`	Table of the status of all locks
`print_sensing_matrix`	Sensing matrix, at DC or at a signal frequency
`print_sidebands`	Table of the sideband frequencies of all modulators
`print_tunings`	Table of the optic tunings
`summary`	Tabular summary of the elements of a model

2.1.1 Comparing and summarising models

`compare_models` function

```
compare_models(model_1: Model | str | Path, model_2: Model | str | Path,
    include_types: None | Sequence[str | type[ModelElement]] = None,
    exclude_types: None | Sequence[str | type[ModelElement]] = ('ReadoutDC', 'ReadoutRF',
        'AmplitudeDetector', 'PowerDetectorDemod1', 'MathDetector', 'DegreeOfFreedom', '
       Cavity'),
    precision: int = 6) -> None
```

Compares multiple Finesse models and displays a table of parameter differences.

Parameters

`models`
: `[list[Model]]` List of Finesse model objects to compare.

`include_all_elements`
: `[bool, optional (default=False)]` If True, include all elements in the comparison (including detectors). If False, only includes components and spaces (detectors excluded).

`element_type`
: `[str or type, optional (default=None)]` If provided, restricts comparison to a specific type of component (e.g. “Mirror”).

`printing`
: [bool, optional (default=True)] If True, prints the comparison table. If False, only returns the `Table` object.

`precision`
: [int, optional (default=6)] Number of significant digits to display for numeric values.

Returns

`Table`
: A finesse `Table` highlighting parameter differences across models. Only parameters with at least one differing value are displayed.

Notes

The naming conventions of components should be consistent across all models being compared. Only components with the same names are compared.

---

summary

function

```
summary(model: Model, add_parameters: bool = True,
    element_type: str | type[ModelElement] | Sequence[str]
                  | Sequence[type[ModelElement]] | None = None,
    excluded_types: tuple[str, ...] | tuple[type[ModelElement], ...] | None = (
        ReadoutDC, ReadoutRF, ReadoutDetectorOutput, AmplitudeDetector,
        PowerDetectorDemod1, MathDetector, DegreeOfFreedom, Cavity),
    print_table: bool = False) -> Table
```

Generates and displays a summary of all or subset of elements in a given model in tabular format. The summary can optionally include the parameters of each component. Additionally, you can filter the summary (using `element_type` and `excluded_types`) to display only specific types of components.

Parameters

`model`
: [Model] The model to summarize.

`add_parameters`
: [bool, optional] Whether to add component parameters and their values, by default True.

`element_type`
: [str | type[ModelElement] | Sequence[str] | Sequence[type[ModelElement]] | None, optional] Element types to include, by default None, which shows all elements.

`excluded_types`
: [tuple[str, ...] | tuple[type[ModelElement], ...], optional] Element type to exclude, by default (`ReadoutDC`, `ReadoutRF`, `ReadoutDetectorOutput`, `AmplitudeDetector`, `PowerDetectorDemod1`, `MathDetector`, `DegreeOfFreedom`, `Cavity`).

`print_table`
: [bool, optional] Whether to print the table, by default False.

Returns

`Table`
: A finesse table.

get_phase_config_status function

```
get_phase_config_status(model: Model) -> PhaseConfigStatus
```

Return the phase configuration state:

Phase	zero_k00	zero_tem00_gouy	
0	False	False	physical
1	True	False	non-physical
2	False	True	physical
3	True	True	non-physical

Parameters

`model`
[finesse.Model] The Finesse model.

2.1.2 Powers

get_powers function

```
get_powers(model: Model, locations: dict[str, str | Node]) -> dict[str, dict]
```

Compute the carrier and total power at selected locations of the model.

The powers are computed from the DC fields of the model, so no detectors need to be present. The carrier power contains only light at zero frequency offset; the total power includes all optical frequencies.

Parameters

`model`
[Model] The model to compute powers for.

`locations`
[dict[str, str or Node]] Mapping of labels to optical node names.

Returns

`dict[str, dict]`
For each label: the node name and the carrier and total power in W. The powers are None if the node is not present in the model.

---

print_powers function

```
print_powers(model: Model, locations: dict[str, str | Node])
```

Print a table with the carrier and total power at selected locations of the model.

The powers are computed from the DC fields of the model, so no detectors need to be present. The carrier power contains only light at zero frequency offset; the total power includes all optical frequencies. If there is no zero-offset frequency, carrier power is labeled as '–' and total power is still shown. Locations whose node is not present in the model are labeled as '–'.

Parameters

`model`
[Model] The model to print powers for.

`locations`
[dict[str, str or Node]] Mapping of labels to optical nodes.

---

print_dc_readout_powers function

```
print_dc_readout_powers(model: Model, readouts: list[str | ReadoutDC] | None = None)
```

Prints a table with the carrier and total power at DC readouts of the model.

The powers are computed from the DC fields at the node each `ReadoutDC` component is attached. The readouts' own outputs are not used. The carrier power contains only the light at the main laser frequency; the total power includes all optical frequencies.

Parameters

`model`
[Model] Model to print the readout powers for.

`readouts`
[list[str], optional] Names of the `ReadoutDC` components to include, e.g. `["B1", "B4"]`. If None (default), all `ReadoutDC` components of the model are used.

Raises

`ValueError`
If a name in `readouts` is not a `ReadoutDC` component of the model.

2.1.3 Degrees of freedom, locks and sensing

---

print_dofs function

```
print_dofs(model: Model,
    dofs: Sequence[str | DegreeOfFreedom] | str | DegreeOfFreedom | None = None)
```

Print a table summarizing degrees of freedom (DOFs) in the model.

The table lists each DOF together with its DC offset and the nodes it drives, including the corresponding drive amplitudes.

If no DOFs are specified, all DOFs defined in `model.dofs` are included.

Parameters

`model`
: [finesse.Model] The Finesse model containing the degrees of freedom.

`dofs`
: [str, DegreeOfFreedom, sequence of (str or DegreeOfFreedom), or None, optional] DOF(s) included in the table. Items may be DOF names or DOF objects. If None (default), all DOFs in `model.dofs` are used.

---

print_lock_status

function

```
print_lock_status(model: Model)
```

Print a formatted table summarizing the status of all locks in a model.

This function iterates over the locks defined in the given model, extracts key parameters for each lock, and displays them in a compact table format.

Parameters

`model`
: [finesse.Model] The finesse model.

---

print_sensing_matrix

function

```
print_sensing_matrix(model: Model,
    dofs: Sequence[str | DegreeOfFreedom] | str | DegreeOfFreedom | None = None,
    readouts: Sequence[str | ReadoutDC | ReadoutRF] | str | ReadoutDC | ReadoutRF | None =
        None,
    highlight_locks: bool = True, frequency: float | None = None, scale: float = 1)
```

Prints the sensing matrix elements for select degrees of freedom and readouts present in the model. If no degrees of freedom and readouts are given, the sensing matrix is computed for all degrees of freedom and readouts defined in the model.

If a frequency is given, the matrix contains the transfer functions from the degrees of freedom to the readout outputs at that signal frequency, printed in the same form as the DC matrix: each complex response is reduced to a signed magnitude, the magnitude of the complex value with the sign of its real part, as in the compass plots. The response at a signal frequency includes all dynamics of the model; for suspended mirrors the response below the suspension resonance frequencies is dominated by radiation pressure coupling and differs from the DC sensing matrix.

Parameters

`dofs`
: [str, DegreeOfFreedom, sequence of (str or DegreeOfFreedom), or None, optional] Degrees of freedom included in the sensing matrix. Items may be DOF names or DOF objects.

If None (default), all degrees of freedom are included.

`readouts`
[str, Readout, sequence of (str or Readout), or None, optional] Readouts included in the sensing matrix. Items may be readout names or readout objects. If None (default), all readouts are included.

`highlight_locks`
[bool, optional] If True (default), the cell of each degree of freedom and error signal pair that is used by an enabled lock of the model is coloured. Has no effect when the model contains no enabled locks.

`frequency`
[float, optional] Signal frequency in Hz at which the responses are evaluated. If None (default), the DC sensing matrix is printed.

`scale`
[float, optional] Overall factor applied to the printed values (default 1). For example, with a frequency given, `scale=1e-12` shows the response per picometre of drive for degrees of freedom defined on mirror displacements. A scale other than 1 is noted above the table.

2.1.4 Sidebands and tunings

`print_sidebands` function

```
print_sidebands(model: Model)
```

Print a formatted table of sideband frequencies for all modulators in a model.

This function retrieves all `Modulator` elements from the provided model, computes their sideband frequencies up to their specified order, and prints the results as a formatted table. Frequencies are displayed in MHz.

Parameters

`model`
[finesse.Model] The Finesse model.

`print_tunings` function

```
print_tunings(model: Model, optics: Sequence[str | Surface] | str | Surface | None = None,
    only_dof: bool = False)
```

Print a table of optic tunings.

This function prints the phase tuning of selected optical surfaces in the model. Tunings are displayed in degrees and converted to an equivalent displacement in picometers. If the tuning is defined symbolically, the symbolic expression is shown. `phi` is the constant offset (with all degrees of freedom set to zero).

Parameters

`model`
[finesse.Model] The Finesse model containing the optical components.

`optics`
: [str, Surface, sequence of (str or Surface), or None, optional] Optical surfaces whose tunings should be printed. Items may be surface names or surface objects. If None (default), all optics are included.

`only_dof`
: [bool, optional] If True and `optics` is None, only optics that appear in at least one DOF drive are included. If False (default), all surfaces in the model are included.

2.2 Plotting: `finesse.plotting`

Plotting functions for Finesse models. This module provides utilities for:

- plotting detector outputs while scanning degrees of freedom;
- plotting error signals derived from the locks of a model;
- compass plots of readout responses at a single signal frequency.

Timo Marchand April 2026

Function	Purpose
`plot_dof_scan`	Detector response while scanning one degree of freedom
`plot_dof_scans`	Grid of DOF scan plots
`plot_error_signals`	Grid of error signal scans, by default from the locks
`plot_compass`	Compass-style polar plot of detector responses
`plot_sensing_matrix`	Compass plots of the sensing of DOFs by RF readouts

2.2.1 Scans of degrees of freedom

`plot_dof_scan` function

```
plot_dof_scan(model: Model, dof: str | DegreeOfFreedom,
    detectors: Sequence[str | Detector | ReadoutDetectorOutput] | str | Detector
               | ReadoutDetectorOutput,
    axis: tuple[float, float, int], square_amplitudes: bool = True,
    logy: bool = False, ax: Axes | None = None) -> Axes
```

Plot the response of one or multiple detectors while scanning a single model degree of freedom (DOF).

This function performs a linear scan of the specified DOF offset over the provided axis range around the DC value of the DOF, runs the simulation, and plots the detector outputs over the scan.

For amplitude detectors and demodulated power detectors with no demodulation phase specified the magnitude of the complex detector output is plotted. If `square_amplitude` is True, the squared magnitude of amplitude detector outputs is plotted instead. All other detector outputs are plotted without modification.

Parameters

`model`
[Model] The simulation model containing the degree of freedom and detector objects.

`dof`
[str or DegreeOfFreedom] The degree of freedom to scan.

`detectors`
[str, Detector, ReadoutDetectorOutput, or sequence of these] The detector(s) to evaluate. These may be readout detector outputs.

`axis`
[tuple[float, float, int]] The scan axis of the degree of freedom, centered at its DC value, in the form (start, stop, npoints).

`square_amplitudes`
[bool, optional] If True (default), the square of the magnitude of the output of amplitude detectors is plotted. If False, the magnitude is plotted.

`logy`
[bool, optional] If True, use a logarithmic scale for the y-axis. Defaults to False.

`ax`
[matplotlib.axes.Axes, optional] Existing matplotlib axes to plot on. If None, a new figure and axes are created.

Returns

`matplotlib.axes.Axes`
The matplotlib axes containing the generated plot.

`plot_dof_scans` function

```
plot_dof_scans(model: Model,
    pairs: Sequence[tuple[str | DegreeOfFreedom,
                          Sequence[str | Detector | ReadoutDetectorOutput] | str | Detector
                          | ReadoutDetectorOutput,
                          tuple[float, float, int]]]
           | tuple[str | DegreeOfFreedom,
                   Sequence[str | Detector | ReadoutDetectorOutput] | str | Detector
                   | ReadoutDetectorOutput,
                   tuple[float, float, int]],
    square_amplitudes: bool = True, logy: bool = False,
    figsize: tuple[float, float] = (10, 6)) -> tuple[Figure, Sequence[Axes]]
```

Create a grid of DOF scan plots for multiple (dof, detector) pairs.

This function generates a grid of scan plots where each subplot shows the response of one or multiple detectors as a single degree of freedom (DOF) is scanned over a specified range around its DC value. It is a convenience wrapper around `plot_dof_scan`, automatically arranging subplots.

Parameters

`model`
[Model] The simulation model containing the degree of freedom and detector objects.

`pairs`
[DofScanPair or Sequence[DofScanPair]] A single scan specification or a sequence of scan specifications. Each specification consists of:

- `dof` (str or DegreeOfFreedom): degree of freedom to scan;
- `detectors` (str, Detector, ReadoutDetectorOutput, or sequence of these): detector(s) to evaluate. These may be readout detector outputs;
- `axis` (tuple[float, float, int]): scan axis of the degree of freedom, centered at its DC value, in the form (start, stop, npoints).

`square_amplitudes`
: [bool, optional] If True (default), the square of the magnitude of the output of amplitude detectors is plotted. If False, the magnitude is plotted.

`logy`
: [bool, optional] If True, use a logarithmic scale for all y-axes. Defaults to False.

`figsize`
: [tuple[float, float], optional] Size of the generated matplotlib figure in inches. Defaults to (10, 6).

Returns

`fig`
: [matplotlib.figure.Figure] The created matplotlib figure containing all subplots.

`axs`
: [Sequence[matplotlib.axes.Axes]] Flattened array of subplot axes.

`plot_error_signals` function

```
plot_error_signals(model: Model,
    pairs: tuple[str | DegreeOfFreedom,
                 Sequence[str | ReadoutDC | ReadoutRF | ReadoutDetectorOutput] | str
                 | ReadoutDC | ReadoutRF | ReadoutDetectorOutput,
                 tuple[float, float, int]]
           | Sequence[tuple[...]] | None = None,
    plot_both_quadratures: bool = False,
    default_axis: tuple[float, float, int] = (-90, 90, 200), logy: bool = False,
    figsize: tuple[float, float] = (10, 6)) -> tuple[Figure, Sequence[Axes]]
```

Create a grid of error signal scan plots for multiple (dof, error signal) pairs.

This function generates a grid of scan plots where each subplot shows the behaviour of an error signal as a single degree of freedom (DOF) is scanned over a specified range. Error signals should be detector outputs of readout components.

By default, the (DOF, error signal) pairs are automatically extracted from the enabled locks in the model.

Parameters

`model`
: [Model] The simulation model containing the degrees of freedom and error signals.

`pairs`
: [ErrorSignalScanPair, Sequence[ErrorSignalScanPair], or None, optional] A single scan specification or a sequence of scan specifications. If None (default), the pairs are extracted from the enabled locks of the model. Each specification consists of:
  - `dof` (str or DegreeOfFreedom): degree of freedom to scan;
  - `readout` (str, ReadoutDC, ReadoutRF, ReadoutDetectorOutput, or sequence of these): readouts measuring the error signal associated with the degree of freedom;

- `axis` (tuple[float, float, int]): scan axis of the degree of freedom, centered at its DC value, in the form (start, stop, npoints).

`plot_both_quadratures`
: [bool, optional] If True, plots both I and Q quadratures of each error signal in one subplot; a single readout detector given as error signal is then expanded to its I and Q pair. Defaults to False. Note that a readout component given as error signal always yields both quadratures; to plot a single quadrature, pass the readout detector instead, for example `readout1_I`.

`default_axis`
: [tuple[float, float, int], optional] Scan axis used for the degrees of freedom extracted from the locks, centered at their DC values, in the form (start, stop, npoints). Only used if `pairs` is None. Defaults to (-90, 90, 200).

`logy`
: [bool, optional] If True, use a logarithmic scale for all y-axes. Defaults to False.

`figsize`
: [tuple[float, float], optional] Size of the generated matplotlib figure in inches. Defaults to (10, 6).

Returns

`fig`
: [matplotlib.figure.Figure] The created matplotlib figure containing all subplots.

`axs`
: [Sequence[matplotlib.axes.Axes]] Flattened array of subplot axes.

2.2.2 Compass plots and sensing matrices

`plot_compass` function

```
plot_compass(angles: Sequence[float], radii: Sequence[float], signals: Sequence[str],
    title: str | None = None, ax: Axes | None = None, legend: bool = True,
    highlight: Sequence[bool] | None = None, min_radius: float | None = None,
    figsize: tuple[float, float] = (8, 6)) -> tuple[Figure, Axes]
```

Make a compass-style polar plot of detector responses.

Each entry of `angles` and `radii` is drawn as a point with an arrow from the origin, labelled with the corresponding entry of `signals`.

Parameters

`angles`
: [sequence of float] Angles in radians.

`radii`
: [sequence of float] Radial values. May be negative, for example for magnitudes in dB below unity. The center of the plot corresponds to the smallest radial value (or zero if all values are positive), and the arrows are drawn from the center, so with dB values the arrow length shows dB above that floor.

`signals`
: [sequence of str] Label for each arrow, shown in the legend. Typically detector names

(no .I or .Q, e.g. ['B2 SB1', 'B4 SB1']).

`title`
: [str, optional] Title of the axes. If None (default), no title is set.

`ax`
: [matplotlib.axes.Axes, optional] Existing matplotlib axes to plot on; must use the polar projection. If None, a new figure and axes are created.

`legend`
: [bool, optional] If True, draw a legend on the axes. Defaults to True.

`highlight`
: [sequence of bool, optional] One flag per arrow. Highlighted arrows are drawn slightly thicker and dashed, with a larger marker with a black edge, for example to mark the readout used by a lock. If None (default), no arrow is highlighted.

`min_radius`
: [float, optional] Lower radial limit of the plot. Values below it are drawn at this floor, so their arrows vanish. If None (default), the limit is the smallest radial value, or zero if all values are positive.

`figsize`
: [tuple[float, float], optional] Size of the generated matplotlib figure in inches, if a new figure is created. Defaults to (8, 6).

Returns

`fig`
: [matplotlib.figure.Figure] The matplotlib figure containing the axes.

`ax`
: [matplotlib.axes.Axes] The polar axes containing the plot.

---

plot_sensing_matrix function

```
plot_sensing_matrix(model: Model, dofs: str | Sequence[str], signals: str | Sequence[str],
    frequency: float, highlight_locks: bool = True, min_gain: float | None = None,
    figsize: tuple[float, float] | None = None) -> tuple[Figure, Sequence[Axes]]
```

Draw compass plots of the sensing of degrees of freedom by RF readouts.

This function computes the transfer functions from each given degree of freedom to the I and Q outputs of the given readouts at a single signal frequency, in one simulation run, and draws the results as a grid of compass plots: one subplot per readout, with one arrow per degree of freedom, matching the arrangement of Finesse's SensingMatrixSolution.plot. The angle of an arrow shows how the response distributes between the I and Q quadratures of the readout, and the radius is the magnitude of the response in dB.

Each quadrature response is a complex number and is reduced to a signed magnitude, the magnitude of the complex value with the sign of its real part. If the chosen frequency is well below all cavity poles and similar features of the instrument response, the responses are real and this reduction is exact. Near such features the phase lag of the response approaches 90 degrees and the sign of a component becomes unreliable. The phase of the complex response is not shown in this plot.

The response includes all dynamics of the model. For suspended mirrors the response below the suspension resonance frequencies is dominated by radiation pressure coupling and differs

from the DC sensing matrix, which holds the mirrors fixed. A frequency above the suspension resonances and below the optical poles is recommended.

Parameters

`model`
: [Model] The simulation model. The function works on a copy; the model passed in is not modified.

`dofs`
: [str or sequence of str] Names of the degrees of freedom driving the responses, one subplot each.

`signals`
: [str or sequence of str] Names of the RF readouts to evaluate. For each name the outputs `{name}.I` and `{name}.Q` are used.

`frequency`
: [float] Signal frequency in Hz at which the transfer functions are evaluated.

`highlight_locks`
: [bool, optional] If True (default), the arrow of each readout and degree of freedom pair that is used by an enabled lock of the model is highlighted with a slightly thicker, dashed arrow and a larger marker with a black edge. Has no effect when the model contains no enabled locks.

`min_gain`
: [float, optional] Lower radial limit of the subplots in dB. Arrows of pairs with a smaller magnitude are drawn with zero length. If None (default), each subplot spans from 0 dB, or from its smallest value, to its largest value.

`figsize`
: [tuple[float, float], optional] Size of the generated matplotlib figure in inches. If None (default), the size is chosen from the grid dimensions.

Returns

`fig`
: [matplotlib.figure.Figure] The created matplotlib figure containing all subplots.

`axs`
: [Sequence[matplotlib.axes.Axes]] Flattened array of polar subplot axes, one per readout.

2.3 Pretuning: `finesse.pretuning`

Function	Purpose
`adjust_cavity_length`	Set the round-trip length of a cavity via one space
`adjust_path_length`	Set the path length between nodes via one space
`adjust_variable_length`	Set a model variable via one space
`optimise_demod_phases`	Optimise demodulation phases (deprecated)

adjust_cavity_length function

```
adjust_cavity_length(model: Model, cavity: str | Cavity, space_to_change: str | Space,
    target_length: float, verbose: bool = False) -> float
```

Adjust the length of a space to achieve a target cavity round-trip length.

This function modifies the length of the specified space so that the cavity's round-trip optical length matches the given numerical target. If the space appears multiple times in the cavity path, the required length correction is distributed evenly across all occurrences.

Parameters

`model`
: [finesse.Model] The Finesse model containing the cavity and space.

`cavity`
: [str or Cavity] Cavity whose length will be set to the target length.

`space_to_change`
: [str or Space] Space component whose length will be adjusted.

`target_length`
: [float] Desired round-trip optical length of the cavity (in meters).

`verbose`
: [bool, optional] If True, prints diagnostic information about the adjustment.

Returns

`delta_l`
: The correction applied to `space_to_change`.

adjust_path_length function

```
adjust_path_length(model: Model, start_node: str | Node,
    end_nodes: list[str | Node] | Node | str, space_to_change: str | Space,
    target_length: float, verbose: bool = False) -> float
```

Adjust the length of a space so that the path length from `start_node` to one or more `end_nodes` matches the numerical target. If multiple `end_nodes` are provided, the average path length is used.

Parameters

`model`
: [finesse.Model] Model of the interferometer.

`start_node`
: [str or Node] Start node of the path.

`end_nodes`
: [str, Node, list of (str or Node)] End node(s) of the path(s). If multiple, the average path length is used.

`space_to_change`
: [str or Space] Space component whose length will be adjusted.

`target_length`
: [float] Desired path length (in meters).

`verbose`
: [bool, optional] If True, prints diagnostic information about the adjustment.

Returns

`delta_l`
: The correction applied to `space_to_change`.

`adjust_variable_length` function

```
adjust_variable_length(model: Model, variable: str | Parameter,
    space_to_change: str | Space, target_length: float, verbose: bool = False) -> float
```

Adjust the length of a space so that a specified model variable reaches a target value. The given variable must depend linearly on the given space.

This function computes the difference between the current value of the specified variable and the desired target value, and the proportionality factor with which the variable depends on the `space_to_change`. It then applies the corresponding correction directly to the length of the selected space.

Parameters

`model`
: [finesse.Model] The Finesse model containing the variable and space.

`variable`
: [str or Parameter] Variable whose length will be set to the target length.

`space_to_change`
: [str or Space] Space component whose length will be adjusted.

`target_length`
: [float] Desired length of the variable (in meters).

`verbose`
: [bool, optional] If True, prints diagnostic information about the adjustment.

Returns

`delta_l`
: The correction applied to `space_to_change`.

`optimise_demod_phases` function

```
optimise_demod_phases(model: Model, *pairs, verbose: bool = True)
    -> OptimiseRFReadoutPhaseDCSolution
```

Optimise demodulation phases for a set of RF readouts.

Deprecated. Use the `OptimiseRFReadoutPhaseDC` action from Finesse directly, for example `model.run(OptimiseRFReadoutPhaseDC("DARM", "B1p_56_I"))`. This function will be removed once the Finesse release with the `verbose` argument for the action is available.

Runs an optimisation of the demodulation phases for the provided readout pairs and returns the resulting optimisation solution. Optionally prints a table comparing the initial and optimised demodulation phases.

Parameters

`model`
: [Model] Finesse model on which the optimisation is performed.

`*pairs`
: Pairs of `DegreeOfFreedom` or `Parameter` and `ReadoutRF` elements, or pairs of their names. If none are provided `OptimiseRFReadoutPhaseDC` will automatically search for `Lock` elements which have `ReadoutRF` error signal and optimise them.

`verbose`
: [bool, optional] If True, print a table showing the initial and optimised demodulation phases for each readout. Default is True.

Returns

`OptimiseRFReadoutPhaseDCSolution`
: Solution returned by the `OptimiseRFReadoutPhaseDC` operation.

2.4 Etalons: `finesse.etalon`

Function	Purpose
`etalon_status`	Status of the etalon formed by an HR and an AR surface
`find_mirror_pairs`	Find HR/AR mirror pairs by name
`get_etalon_info`	Etalon information for `print_model_state`
`test_all_mirrors`	Print the etalon status of several mirror pairs

`etalon_status` function

```
etalon_status(HR: Mirror, AR: Mirror, verbose: bool = False) -> EtalonStatus
```

Check if two mirror surfaces form an etalon with linked tunings.

An etalon is formed when both mirror surfaces are aligned. This function checks alignment and whether the tuning parameters are symbolically linked.

Parameters

`HR`
: [finesse.components.mirror.Mirror] High-reflectivity (HR) mirror surface.

`AR`
: [finesse.components.mirror.Mirror] Anti-reflection (AR) mirror surface.

`debug`
: [bool, optional] If True, print diagnostic messages. Default is False.

Returns

`int`
Status code:

- 0 — AR surface is misaligned (no etalon formed);
- 1 — surfaces form etalon but tunings are not linked;
- 2 — surfaces form etalon and tunings are symbolically linked.

Example

```
>>> status = etalon_status(model.NI, model.NIAR, debug=True)
Etalon check: NI - NIAR: tunings are not linked
>>> status
1
```

`find_mirror_pairs` function

```
find_mirror_pairs(model: Model, verbose: bool = False) -> tuple[tuple[str, str], ...]
```

Find all mirror pairs in the model following the 'NE' 'NEAR' convention, looking for mirrors with identical names plus a 'AR' suffix (case insensitive).

Parameters

`model`
[`Model`] Model to look for mirrors.

`verbose`
[`bool, optional`] Print results for every mirror checked, by default False.

Returns

`tuple[tuple[str, str], ...]`
A collection of mirror pair names.

`get_etalon_info` function

```
get_etalon_info(model: Model, *, mirror_pairs: None | Sequence[tuple[str, str]] = None,
    verbose: bool = False) -> tuple[EtalonStatus, str]
```

Return etalon information formatted specifically for `print_model_state`.

Returns

`tuple[str, str]`
`overall_state`: "None", "Free", or "Link"; `summary_line`: formatted string with detailed etalon info.

`test_all_mirrors` function

```
test_all_mirrors(model: Model, mirror_pairs: dict[str, list[str]]) -> None
```

Test etalon status for multiple mirror pairs.

Checks each pair of mirrors and prints status with visual indicators.

Parameters

`model`
[finesse.Model] Finesse model containing the mirrors.

`mirror_pairs`
[dict] Dictionary with keys as identifiers and values as `[HR_name, AR_name]` lists.

Example

```
>>> pairs = {'NI': ['NI', 'NIAR'], 'NE': ['NE', 'NEAR']}
>>> test_all_mirrors(base, pairs)
Etalon check: NI - NIAR: AR surface is misaligned
Etalon check: NE - NEAR: tunings are not linked
```

2.5 Status types: finesse.status

The status classes are integer enumerations used in the model summaries. Each provides a colour and a short and a long description.

BaseStatus class

```
class BaseStatus(new_class_name, /, names, *, module=None, qualname=None, type=None,
                 start=1, boundary=None)
```

Bases: IntEnum

```
abstract property color:  ANSICode

abstract property long_desc:  str

abstract property short_desc:  str
```

EtalonStatus class

```
class EtalonStatus(*values)
```

Bases: BaseStatus

```
MISALIGNED = 0

UNLINKED = 1

GOOD = 2

property color:  ANSICode
```

property long_desc: str

property short_desc

PhaseConfigStatus

class

```
class PhaseConfigStatus(*values)
```

Bases: BaseStatus

k00_False_tem00_gouy_False = 0

k00_True_tem00_gouy_False = 1

k00_False_tem00_gouy_True = 2

k00_True_tem00_gouy_True = 3

property color: ANSICode

property long_desc

property short_desc

StandardStatus

class

```
class StandardStatus(*values)
```

Bases: BaseStatus

ERROR = 0

WARNING = 1

OK = 2

property color

property long_desc

property short_desc

ModelStateItem class

```
class ModelStateItem(names: tuple[str, ...], required: int,
                     _status: ifotools.finesse.status.BaseStatus | None = None,
                     summary: str | None = None)
```

Bases: object

names: tuple[str, ...]

required: int

summary: str | None = None

property count: int

property details: str

property status: BaseStatus

get_status function

```
get_status(count: int, required: int) -> StandardStatus
```

2.6 Utilities: finesse.utils

Function	Purpose
load_model	Load a model from KatScript, a path or a model
get_model_elements	Elements of a model, filtered by type
get_lock_pairs	DOF and error signal pairs of the enabled locks
resolve_dof_scan_pair_detector	Resolve a detector for a DOF scan plot
resolve_dof_scan_pairs	Normalise DOF scan specifications
resolve_error_signal_scan_pairs	Normalise error signal scan specifications

load_model function

```
load_model(model: Model | Path | str) -> Model
```

Load a model from either a KatScript string, a filesystem path, or just returns the model if it is already a model.

Parameters

model
: [Model | Path | str] KatScript file path, KatScript string or Finesse model.

Returns

`Model`
Finesse Model.

Raises

`ValueError`
When the input can not be converted to a Finesse model.

---

get_model_elements function

```
get_model_elements(model: Model,
    include_types: None | Sequence[str | type[ModelElement]] = None,
    exclude_types: None | Sequence[str | type[ModelElement]] = None)
    -> tuple[ModelElement, ...]
```

Get all elements in model, optionally filtered by element type.

Parameters

`model`
[Model] Model to get elements from.

`include_types`
[None | Sequence[str | type[ModelElement]], optional] Which types of elements to include, by default None, which includes everything.

`exclude_types`
[None | Sequence[str | type[ModelElement]], optional] Which type of elements to exclude, by default None, which excludes nothing.

Returns

`tuple[ModelElement, ...]`
All elements in model, optionally filtered by element type.

---

get_lock_pairs function

```
get_lock_pairs(model: Model) -> dict[str, str]
```

Return the degree of freedom and error signal pairs of the enabled locks.

Parameters

`model`
[Model] Model to read the locks from.

Returns

`dict[str, str]`
Maps the name of the degree of freedom controlled by each enabled lock to the name of the detector used as its error signal.

resolve_dof_scan_pair_detector function

```
resolve_dof_scan_pair_detector(model: Model,
    detector: str | Detector | ReadoutDetectorOutput) -> Detector | ReadoutDetectorOutput
```

Resolve and validate a detector for use in a DOF scan plot.

Parameters

`model`
[Model] Model containing the detector.

`detector`
[str, Detector or ReadoutDetectorOutput] Detector object or name identifying the detector to use.

Returns

`Detector or ReadoutDetectorOutput`
The resolved detector object.

resolve_dof_scan_pairs function

```
resolve_dof_scan_pairs(model: Model, pairs: DofScanPair | Sequence[DofScanPair])
    -> tuple[list[DegreeOfFreedom], list[list[Detector | ReadoutDetectorOutput]],
             list[tuple[float, float, int]]]

# DofScanPair = tuple[str | DegreeOfFreedom,
#                     Sequence[str | Detector | ReadoutDetectorOutput] | str | Detector
#                     | ReadoutDetectorOutput,
#                     tuple[float, float, int]]
```

Validate and normalize DOF scan specifications.

This helper function converts the user-facing `pairs` argument accepted by `plot_dof_scans` into a normalized internal representation. Degree-of-freedom and detector names are resolved using the model, and all inputs are validated.

Parameters

`model`
[Model] Model used to resolve degree-of-freedom and detector names.

`pairs`
[DofScanPair or Sequence[DofScanPair]] A single scan specification or a sequence of scan specifications. Each specification consists of:

- `dof` (str or DegreeOfFreedom): degree of freedom to scan;
- `detectors` (str, Detector, ReadoutDetectorOutput, or sequence of these): detector(s) to evaluate. These may be readout detector outputs;
- `axis` (tuple[float, float, int]): scan axis of the degree of freedom, centered at its DC value, in the form (start, stop, npoints).

Returns

`dofs`
[list[DegreeOfFreedom]] Resolved degree-of-freedom objects.

`detectors`
[list[list[Detector | ReadoutDetectorOutput]]] Resolved detector or readout detector output objects for each scan. Each entry is a list, even if a single detector or readout detector output was supplied.

`scans`
[list[tuple[float, float, int]]] Scan ranges corresponding to each degree of freedom.

---

resolve_error_signal_scan_pairs

function

```
resolve_error_signal_scan_pairs(model: Model,
    pairs: ErrorSignalScanPair | Sequence[ErrorSignalScanPair] | None = None,
    plot_both_quadratures: bool = False)
    -> tuple[list[DegreeOfFreedom], list[list[ReadoutDetectorOutput]],
             list[tuple[float, float, int]]]

# ErrorSignalScanPair = tuple[str | DegreeOfFreedom,
#                             Sequence[str | ReadoutDC | ReadoutRF | ReadoutDetectorOutput]
#                             | str | ReadoutDC | ReadoutRF | ReadoutDetectorOutput,
#                             tuple[float, float, int]]
```

Validate and normalize error signal scan specifications.

This helper function converts the user-facing `pairs` argument accepted by `plot_error_signals` into a normalized internal representation. Degree-of-freedom and detector names are resolved using the model, and all inputs are validated.

Parameters

`model`
[Model] Model used to resolve degree-of-freedom and detector names.

`pairs`
[ErrorSignalScanPair or Sequence[ErrorSignalScanPair]] A single scan specification or a sequence of scan specifications. Each specification consists of:

- `dof` (str or DegreeOfFreedom): degree of freedom to scan;
- `readout` (str, ReadoutDC, ReadoutRF, ReadoutDetectorOutput, or sequence of these): readouts or ReadoutDetectorOutputs measuring the error signal associated with the degree of freedom;
- `axis` (tuple[float, float, int]): scan axis of the degree of freedom, centered at its DC value, in the form (start, stop, npoints).

Returns

`dofs`
[list[DegreeOfFreedom]] Resolved degree-of-freedom objects.

`readout_detector_outputs`
[list[list[ReadoutDetectorOutput]]] Resolved readout detector output objects for each scan.

`scans`
[list[tuple[float, float, int]]] Scan ranges corresponding to each degree of freedom.

CHAPTER 3

Virgo tools: `ifotools.virgo`

3.1 Model information: `virgo.model_info`

Function	Purpose
`get_model_state`	Structured information about the model state
`print_model_state`	Coloured summary of the model state
`get_powers`	Carrier and total power at the standard Virgo locations
`print_powers`	Table of these powers
`get_qnls`	Quantum noise limited sensitivity
`print_lengths_frequencies`	Table of important lengths and frequencies

3.1.1 Model state

`get_model_state` function

```
get_model_state(model: Model) -> dict[str, ModelStateItem]
```

Collect and return structured information about the current model state.

Parameters

`model`
[`Model`] Model to collect information from.

Returns

`dict[str, dict[str, Any]]`
State for various topics in the model.

`print_model_state` function

```
print_model_state(model: Model, *, print_table: bool = True, verbose: bool = False) -> str
```

Prints a coloured summary of the model state.

Parameters

`model`
[`Model`] Finesse model to inspect.

`print_table`
: [bool, optional] If true, prints the overview that is returned.

`verbose`
: [bool, optional] If True, add detailed information for each block.

Returns

`str`
: Overview (with ANSI colors) of the Virgo model status.

3.1.2 Powers

---

get_powers function

```
get_powers(model: Model, locations: dict[str, str] = {'Dark fringe': 'B1.p1.i',
    'North arm': 'NE.p1.o', 'PRC': 'BS.p1.i', 'West arm': 'WE.p1.o'}) -> dict[str, dict]
```

Compute the carrier and total power at selected locations of the model.

The powers are computed from the DC fields of the model, so no detectors need to be present. The carrier power contains only light at zero frequency offset; the total power includes all optical frequencies, in particular the RF sidebands.

Parameters

`model`
: [Model] Virgo model to compute powers for.

`locations`
: [dict[str, str], optional] Mapping of labels to optical node names. If None (default), the north and west arm cavities, the PRC, and the dark fringe of the standard Virgo model are used.

Returns

`dict[str, dict]`
: For each label: the node name and the carrier and total power in W. The powers are None if the node is not present in the model. Carrier power is also None if the solution has no zero-offset frequency.

---

print_powers function

```
print_powers(model: Model, locations: dict[str, str] = {'Dark fringe': 'B1.p1.i',
    'North arm': 'NE.p1.o', 'PRC': 'BS.p1.i', 'West arm': 'WE.p1.o'})
```

Print a table with the carrier and total power at selected locations of the model.

The powers are computed from the DC fields of the model, so no detectors need to be present. The carrier power contains only light at zero frequency offset; the total power includes all optical frequencies, in particular the RF sidebands. Locations whose node is not present in the model are labeled as '–'. If there is no zero-offset frequency, only carrier power is labeled as '–'.

Parameters

`model`
: [Model] Virgo model to print powers for.

`locations`
: [dict[str, str], optional] Mapping of labels to optical node names. If None (default), the north and west arm cavities, the PRC, and the dark fringe of the standard Virgo model are used.

3.1.3 Sensitivity, lengths and frequencies

---

get_qnls function

```
get_qnls(model: Model, axis: tuple[float, float, int] = (5, 5000, 100),
    arms: tuple[str | Space, str | Space] = ('LN', 'LW'),
    readout: str | ReadoutDC | OpticalNode = 'B1.p1.i', nsr: bool = True) -> ArraySolution
```

Compute the Quantum Noise Limited Sensitivity (QNLS) for a given model.

This function creates a deep copy of the input model, applies differential modulation signals to the specified interferometer arms, and evaluates both the full QNLS (including radiation pressure) and the shot-noise-limited sensitivity for each frequency in the given axis. This can be done at any readout and node in the model.

Parameters

`model`
: [Model] The interferometer model to be used for the QNLS calculation.

`axis`
: [tuple of (float, float, int), optional] Frequency sweep parameters in the form (start, stop, number of points). Defaults to (5, 5000, 100).

`arms`
: [tuple of (str | Space, str | Space), optional] Space components describing the two interferometer arms to be differentially modulated. This is the space between the input and end mirror. The second arm is modulated with a 180-degree phase shift. Defaults to ("LN", "LW").

`readout`
: [str or ReadoutDC or OpticalNode] Readout or node at which the QNLS is computed. Defaults to "B1.p1.i".

`nsr`
: [bool, optional] If True, calculate the noise-to-signal ratio rather than the absolute noise value. Defaults to True.

Returns

`result`
: [finesse.solutions.ArraySolution] Solution object containing the frequency axis and noise amplitude spectral densities, accessible via:
  - `"NSR_with_RP"`: quantum noise including radiation pressure;
  - `"NSR_without_RP"`: shot-noise-limited sensitivity.

print_lengths_frequencies function

```
print_lengths_frequencies(model: Model, *, arms=('LN.L', 'LW.L'),
    central_ifo=('lpr', 'ln', 'lw', 'lPRC', 'lSRC', 'lMI', 'lSchnupp'),
    fsr=('fsrN', 'fsrW', 'fsrPRC', 'fsrSRC'),
    fre_match=('f1_arm', 'f1_SRC', 'f1_PRC'), verbose: bool = False)
```

Prints a table with various important lengths and frequencies from the Virgo interferometer. This function relies on variables with specific names being present in the model. Lengths of spaces can also be given directly. If a variable can not be found, it will be labeled as '–' in the table. You can adjust the names of the variables this function looks for with the keyword arguments described below.

Parameters

`model`
: [Model] Virgo model to print lengths and frequencies for.

`arms`
: [tuple, optional] Variables describing the lengths of the north and west arm, by default ("LN.L", "LW.L").

`central_ifo`
: [tuple, optional] Variable names for lengths in the central interferometer, by default ("lpr", "ln", "lw", "lPRC", "lSRC", "lMI", "lSchnupp").

`fsr`
: [tuple, optional] The free spectral range for the north/west arm and the power/signal recycling cavities, by default ("fsrN", "fsrW", "fsrPRC", "fsrSRC").

`fre_match`
: [tuple, optional] *(description pending)*, by default ("f1_arm", "f1_SRC", "f1_PRC").

`verbose`
: [bool, optional] Whether to add a column with the symbolic values of these variables, by default False.

3.2 Plotting: virgo.plotting

plot_qnls function

```
plot_qnls(model: Model, axis: tuple[float, float, int] = (5, 5000, 100),
    arms: tuple[str | Space, str | Space] = ('LN', 'LW'),
    readout: str | ReadoutDC | OpticalNode = 'B1.p1.i', nsr: bool = True,
    shot_noise_only: bool = False, ax: Axes | None = None) -> Axes
```

Plot the quantum noise limited sensitivity (QNLS) of an interferometer model.

This function computes the noise spectral density using `get_qnls` and plots the amplitude spectral density (ASD) versus signal frequency at any readout or node in the model. The shot-noise limited sensitivity is always plotted, and the full quantum noise (including radiation pressure) is optionally included.

Parameters

`model`
[Model] The interferometer model to be used for the QNLS calculation.

`axis`
[tuple of (float, float, int), optional] Frequency sweep parameters in the form (start, stop, number of points). Defaults to (5, 5000, 100).

`arms`
[tuple of (str | Space, str | Space), optional] Space components describing the two interferometer arms to be differentially modulated. This is the space between the input and end mirror. The second arm is modulated with a 180-degree phase shift. Defaults to ("LN", "LW").

`readout`
[str or ReadoutDC or OpticalNode, optional] Readout or node at which the QNLS is computed. Defaults to "B1.p1.i".

`nsr`
[bool, optional] If True, calculate the noise-to-signal ratio rather than the absolute noise value. Defaults to True.

`shot_noise_only`
[bool, optional] If True, only the shot-noise-limited sensitivity is plotted. If False, both shot noise and full quantum noise (including radiation pressure) are plotted. Defaults to False.

`ax`
[matplotlib.axes.Axes, optional] Existing matplotlib axes to plot on. If None, a new figure and axes are created.

Returns

`ax`
[matplotlib.axes.Axes] The axes containing the plotted quantum-noise limited sensitivity. If `ax` was provided, the plot is added to that axes and the same object is returned. Otherwise a new axes is created and returned.

3.3 Pretuning: `virgo.pretuning`

Function	Purpose
`calculate_recycling_cavity_length`	Optimal recycling cavity length
`adjust_prc_length`	Set the PRC length
`adjust_src_length`	Set the SRC length
`pretune`	Pretune the model

`calculate_recycling_cavity_length` function

```
calculate_recycling_cavity_length(model: Model, f6MHz_modulator: Modulator | str,
    verbose: bool = False) -> float
```

Calculate the optimal recycling cavity length as

$$0.5 \cdot c/(2f_6)$$

where c is the speed of light and f_6 is the modulation frequency of the specified modulator. See TDR 2.3 (VIR–0128A–12).

Parameters

`model`
: [finesse.Model] The Finesse Model.

`f6MHz_modulator`
: [Modulator | str] The modulator to get the frequency from.

`verbose`
: [bool, optional] Verbosity, by default False.

Returns

`float`
: The optimal cavity length in meters.

---

adjust_prc_length

function

```
adjust_prc_length(model: Model, input_node: str | Node = 'PR.p2.o',
    output_nodes: tuple[str | Node, ...] | Node | str = ('WI.p1.i', 'NI.p1.i'),
    space_to_change: str | Space = 'lPR_POP',
    f6MHz_modulator: str | Modulator = 'eom6',
    numerical_target: float | None = None, verbose: bool = False)
```

Adjust the Power Recycling Cavity (PRC) length to satisfy a resonance condition.

This function adjusts the specified space length so that the optical path length from `input_node` to `output_nodes` matches a given numerical target. If no numerical target is provided, it is computed from the modulation frequency of the specified modulator (typically EOM6).

Parameters

`model`
: [finesse.Model] The Finesse model containing the PRC components.

`input_node`
: [str or Node, optional] Input node of the PRC path. Default is “PR.p2.o”.

`output_nodes`
: [str, Node, list of (str or Node), optional] Output node(s) defining the end of the PRC path. If multiple nodes are given, the average optical path length is used. Defaults to [“WI.p1.i”, “NI.p1.i”].

`space_to_change`
: [str or Space, optional] The space component whose length will be adjusted. Default is the space between power recycling mirror and pickoff plate, “lPR_POP”.

`f6MHz_modulator`
: [str or Modulator, optional] Modulator used to compute the resonance target from its modulation frequency.

`numerical_target`
: [float, optional] Desired optical path length in meters. If None, it is calculated as $0.5 \cdot c/(2f_6)$ where c is the speed of light and f_6 is the modulation frequency of the specified modulator. See TDR 2.3 (VIR–0128A–12).

`verbose`
[bool, optional] If True, prints diagnostic information about the computed target and applied length adjustment.

---

adjust_src_length

function

```
adjust_src_length(model: Model, input_node: str | Node = 'SR.p1.o',
    output_nodes: tuple[str | Node, ...] | Node | str = ('WI.p1.i', 'NI.p1.i'),
    space_to_change: str | Space = 'lsr',
    f6MHz_modulator: str | Modulator = 'eom6',
    numerical_target: float | None = None, verbose: bool = False)
```

Adjust the Signal Recycling Cavity (SRC) length to satisfy a resonance condition.

This function adjusts the specified space length so that the optical path length from `input_node` to `output_nodes` matches a given numerical target. If no numerical target is provided, it is computed from the modulation frequency of the specified modulator (typically EOM6).

Parameters

`model`
[finesse.Model] The Finesse model containing the SRC components.

`input_node`
[str or Node, optional] Input node of the SRC path. Default is "SR.p1.i".

`output_nodes`
[str, Node, list of (str or Node), optional] Output node(s) defining the end of the SRC path. If multiple nodes are given, the average optical path length is used. Defaults to ["WI.p1.i", "NI.p1.i"].

`space_to_change`
[str or Space, optional] The space component whose length will be adjusted. Default is the space between signal recycling mirror and beam splitter, "lsr".

`f6MHz_modulator`
[str or Modulator, optional] Modulator used to compute the resonance target from its modulation frequency.

`numerical_target`
[float, optional] Desired optical path length in meters. If None, it is calculated as $0.5 \cdot c/(2f_6)$ where c is the speed of light and f_6 is the modulation frequency of the specified modulator. See TDR 2.3 (VIR–0128A–12).

`verbose`
[bool, optional] If True, prints diagnostic information about the computed target and applied length adjustment.

---

pretune

function

```
pretune(model: Model, verbose: bool = False)
```

3.4 Pretuning code generator: virgo.pretuning_generator

generate_pretuning_code function

```
generate_pretuning_code(modulators: tuple[str, str, str] = ('eom1', 'eom2', 'eom3'),
    prc: tuple[str, str, str, str] = ('PR', 'PRAR', 'PRCL', 'CAR_AMP_BS'),
    src: tuple[str, str, str, str] = ('SR', 'SRAR', 'SRCL', 'B1_DC'),
    north_arm: tuple[str, str, str, str] = ('NI', 'NE', 'NE_z', 'B7_DC'),
    west_arm: tuple[str, str, str, str] = ('WI', 'WE', 'WE_z', 'B8_DC'),
    michelson: tuple[str, str] = ('B1_DC', 'MICH'), verbose: bool = False) -> str
```

Generates a piece of Python code that you can copy-paste into your notebook to pre-tune your Virgo model. Uses some 'default' names for model elements that you can adjust after copying, or by using some of the arguments.

Parameters

`modulators`
: `[tuple[str, str, str], optional]` Modulator names, by default ("eom1", "eom2", "eom3").

`prc`
: `[tuple[str, str, str, str], optional]` Power recycling mirror, anti-reflective mirror, common length DOF name and detector name, by default ("PR", "PRAR", "PRCL", "CAR_AMP_BS").

`src`
: `[tuple[str, str, str, str], optional]` Signal recycling mirror, anti-reflective mirror, common length DOF name and detector name, by default ("SR", "SRAR", "SRCL", "B1_DC").

`north_arm`
: `[tuple[str, str, str, str], optional]` Input mirror, end mirror, end mirror tuning DOF and detector, by default ("NI", "NE", "NE_z", "B7_DC").

`west_arm`
: `[tuple[str, str, str, str], optional]` Input mirror, end mirror, end mirror tuning DOF and detector, by default ("WI", "WE", "WE_z", "B8_DC").

`dark_fringe`
: `[tuple[str, str], optional]` Output detector and Michelson degree of freedom, by default ("B1_DC", "MICH").

`verbose`
: `[bool, optional]` Whether to print the pre-tuning code.

Returns

`str`
: The pre-tuning code.

generate_imports function

```
generate_imports() -> str
```

CHAPTER 4

Mirror maps: ifotools.mirror_maps

4.1 The ET mirror data container: mirror_maps.container

The ET mirror data container: reading and writing
Andreas Freise, 01.10.2026

Functions for files of the ET mirror data container format, specified in the ET note ET-0774A-26, "The ET mirror data container: schema specification". An HDF5 file holds data trees such as `nominal`, `measurements`, `processed` and `recommended`, with the common structure `<tree>/<surface>/<quantity>/<label>/`. The group at the end of such a path is an *entry*. It stores a scalar as the attribute `value` with a `unit`, a map (`x`, `y` and `Z`, with the unit on `Z`), local samples in a `samples` subgroup, or a set of named attributes such as the substrate geometry.

Main functions

Function	Purpose
Reading	
`info`	Print a listing of a file: root attributes, history, one line per entry with its content and attributes
`entries`	List the entry paths, filtered by tree, surface, quantity
`read_map`	Read a map entry: x, y, Z and the attributes
`read_value`	Read a scalar entry: value, unit and the attributes
`read_attrs`	Read all attributes of an entry
`read_samples`	Read the local samples of an entry
`validate`	Check a file against the schema, return the problems
Writing	
`create_file`	Create a file with its root attributes and a new uuid
`write_map`	Write a map entry
`write_value`	Write a scalar entry
`write_attrs`	Write named attributes to one or several entries
`set_recommended`	Point `recommended/<surface>/<quantity>` at another entry
`remove_entry`	Remove an entry, or a group with its entries
`append_history`	Record a modification in `file_history` and `revision`

Technical notes

- Every function takes a file path or an open `h5py.File`.
- The functions encode the schema rules and nothing else. Anything beyond the required minimum is optional: extra attributes, datasets and groups are passed through, listed by

`info()` and never reported by `validate()`.

- The writers and `remove_entry()` do not update the history. Call `append_history()` explicitly to do so, e.g. once per significant modification of an existing file.
- `set_recommended()` makes a soft link for a map and a copy with a `source` attribute for a scalar or an attribute set.
- A group is an entry when it holds a `value` attribute, a dataset `Z` or a `samples` group, or when it is a leaf group holding attributes or datasets. The last clause covers the geometry entries of `substrate` and `ingot` and the derived-product groups. Groups above an entry are containers; attributes found on them are listed but have no meaning to the schema. Nothing below an entry is an entry.

Requires: `numpy`, `h5py`.

4.1.1 Reading and inspecting

`info` function

```
info(file: str | Path | File, print_out: bool = True) -> str
```

List the content of a file in a readable form.

The listing starts with the root attributes and the history, then has one block per entry: the entry path as header, the content (map shape and unit, number of samples, further datasets) and the attributes, one per line and aligned. Free text (`summary`, `comment`) is quoted, and a history of more than four entries shows its first two and its last. Soft links are shown with their target, attributes found on container groups are shown where they sit. Nothing is an error here; a file in another layout is listed as far as it goes.

Parameters

`file`
: Path or open `h5py.File`.

`print_out`
: Print the listing (default) in addition to returning it.

Returns

The listing as one string.

`entries` function

```
entries(file: str | Path | File, tree: str | None = None, surface: str | None = None,
    quantity: str | None = None) -> list[str]
```

List the entry paths of a file.

Parameters

`file`
: Path or open `h5py.File`.

`tree`
: Restrict to one of `nominal`, `measurements`, `processed`, `recommended`.

`surface`
: Restrict to one of `s1`, `s2`, `substrate`, `ingot`.

`quantity`
: Restrict to one quantity name, e.g. `surface_height`.

Returns

Entry paths without leading slash, in file order, trees in schema order. Soft-linked entries of `recommended/` are listed by their own path.

Example

```
>>> entries("Virgo_EM01.h5", tree="processed", quantity="surface_height")
['processed/s1/surface_height/280mm', 'processed/s1/surface_height/350mm']
```

`is_entry` function

```
is_entry(group: Group) -> bool
```

Tell whether a group is an entry of the container.

An entry holds a `value` attribute, a dataset `Z` or a `samples/` group, or it is a leaf group that holds attributes or datasets.

`read_map` function

```
read_map(file: str | Path | File, path: str) -> tuple[ndarray, ndarray, ndarray, dict[str,
       Any]]
```

Read the map of an entry.

Works for any entry holding the datasets `x`, `y` and `Z`, including `recommended/<surface>/surface_height` through its soft link.

Returns

`(x, y, Z, attrs)` with `x` and `y` in m, `Z` of shape `(len(y), len(x))` and `attrs` the attributes of the entry group, with `attrs["unit"]` set to the unit of `Z`.

Raises

`KeyError`
: If the entry holds no dataset `Z`.

Example

```
>>> x, y, Z, attrs = read_map("reference_map.h5",
...                           "processed/s1/surface_height/350mm")
>>> attrs["unit"], attrs["aperture"]
('nm', 0.35)
```

read_value

function

```
read_value(file: str | Path | File, path: str) -> tuple[Any, str, dict[str, Any]]
```

Read the scalar of an entry.

Returns

`(value, unit, attrs)`: the `value` attribute, its `unit` and the dictionary of all attributes of the entry, which holds `uncertainty`, `tolerance`, `source` and the conditions where present.

Raises

`KeyError`
: If the entry holds no `value` attribute.

Example

```
>>> roc, unit, attrs = read_value("Virgo_EM01.h5", "recommended/s1/roc")
>>> attrs["source"]
'/measurements/s1/roc/lma_2015-05'
```

read_attrs

function

```
read_attrs(file: str | Path | File, path: str) -> dict[str, Any]
```

Read all attributes of an entry group as a dictionary.

Numpy scalars are converted to Python numbers and strings; arrays stay numpy arrays. For an entry of `recommended/` the dictionary includes `source`.

Example

```
>>> read_attrs("Virgo_EM01.h5", "measurements/substrate/geometry")
{'material': 'fused silica', 'diameter': 0.35, ..., 'producer': 'LMA'}
```

read_samples

function

```
read_samples(file: str | Path | File, path: str) -> dict[str, ndarray]
```

Read the local samples of an entry as a dictionary of arrays.

`path` is the entry or its `samples/` group. String datasets, such as the zone labels, come back as arrays of `str`.

Example

```
>>> s = read_samples("Virgo_EM01.h5",
...                  "measurements/s1/roughness/newview_2024-09")
>>> s["rms"][s["zone"] == "A"].mean()
```

validate

function

```
validate(file: str | Path | File) -> list[str]
```

Check a file against the required fields and invariants of the schema.

Checked are the required root attributes, the schema name, the form of `uuid`, the pairing and consistency of `revision` and `file_history`; for every entry the datasets `x`, `y`, `Z` and the shape of `Z`, a `unit` on every floating-point dataset and beside every `value`; for every copied entry of `recommended/` a `source` that exists and, where both hold a value, agrees with it; and that every soft link has a target. Optional content is never reported.

Returns

A list of problems as strings, each starting with the path concerned. An empty list means the file is valid.

Example

```
>>> validate("reference_map.h5")
[]
```

4.1.2 Creating and writing

create_file

function

```
create_file(path: str | Path, name: str, project: str, author: str,
    summary: str | None = None, date: str | None = None, created_by: str | None = None,
    overwrite: bool = False, **root_attrs: Any) -> str
```

Create a new container file with its root attributes.

Writes `name`, `project`, `schema`, `schema_version`, a fresh version 4 `uuid`, `revision = 1`, the creation entry of `file_history`, `created_by` and, if given, `summary` and any further root attributes (`type`, `surface1`, `surface2`, ...).

Parameters

`path`
: File to create.

`name`
: Short mirror identifier, e.g. “EM01”.

`project`
: Top-level programme, e.g. “Virgo”.

`author`
: Author of the creation entry in the history.

`summary`
: Free-text description of the file contents.

`date`
: ISO date of creation; today by default.

`created_by`
: Software that wrote the file; ifotools by default.

`overwrite`
Replace an existing file.

`**root_attrs`
Further root attributes.

Returns

The uuid of the new file.

Example

```
>>> create_file("Virgo_EM01.h5", "EM01", "Virgo", "A. Freise",
...              summary="LMA characterisation 2015", type="end")
```

`write_map` function

```
write_map(file: str | Path | File, path: str, x: ndarray, y: ndarray, Z: ndarray,
    unit: str = 'nm', producer: str | None = None, date: str | None = None,
    comment: str | None = None, overwrite: bool = False,
    compression: str | None = 'gzip', **attrs: Any) -> None
```

Write a map entry: the datasets `x`, `y` (m) and `Z` with its unit.

`Z` must have the shape `(len(y), len(x))`, row-major as the schema requires. On a uniformly spaced grid the attributes `pixel_size` and `size` are filled in unless given. Further attributes (`aperture`, `document`, `wavelength`, ...) are passed through.

Parameters

`file`
Path or open `h5py.File`, opened for writing.

`path`
Entry path, e.g. "processed/s1/surface_height/350mm". Groups along the path are created as needed.

`x, y`
Coordinate axes in m.

`Z` Map values of shape `(len(y), len(x))`, NaN outside the data.

`unit`
Unit of `Z`; "nm" for surface height, "ppm" for reflectivity.

`producer, date, comment`
Provenance attributes of the entry.

`overwrite`
Replace a map already stored at `path`.

`compression`
h5py compression for `Z`; None for none.

`**attrs`
Further attributes of the entry group.

Example

```
>>> write_map("Virgo_EM01.h5", "processed/s1/surface_height/350mm",
...           x, y, Z, producer="A. Freise", date="2026-04-10",
...           aperture=0.35, comment="extrapolated from the 280mm map")
```

write_value

function

```
write_value(file: str | Path | File, path: str, value: Any, unit: str,
    uncertainty: Any = None, tolerance: Any = None, producer: str | None = None,
    date: str | None = None, comment: str | None = None, overwrite: bool = False,
    **attrs: Any) -> None
```

Write a scalar entry: the attributes `value` and `unit` and their company.

Parameters

`file`
: Path or open `h5py.File`, opened for writing.

`path`
: Entry path, e.g. “measurements/s1/roc/lma_2015-05”, or without label where the schema allows it, “nominal/s1/roc”.

`value`
: The number.

`unit`
: Its unit, e.g. “m” or “ppm”.

`uncertainty`
: `[lower, upper]` for a measurement.

`tolerance`
: `[lower, upper]` for a design value.

`producer, date, comment`
: Provenance attributes of the entry.

`overwrite`
: Replace a value already stored at `path`.

`**attrs`
: Further attributes, e.g. `wavelength=1.064e-6`, `document="VIR-0269A-15"`.

Example

```
>>> write_value("Virgo_EM01.h5", "measurements/s1/roc/lma_2015-05",
...             1695.0, "m", uncertainty=[-10.0, 5.0], extent=0.15,
...             producer="LMA", document="VIR-0269A-15", date="2015-05")
```

write_attrs

function

```
write_attrs(file: str | Path | File, paths: str | list[str], overwrite: bool = False,
    **attrs: Any) -> None
```

Write named attributes to one entry or to several entries at once.

Used for an entry made of attributes, such as the substrate geometry, and for attributes shared by several entries, such as the producer, document and date of one measurement campaign.

Parameters

`file`
: Path or open `h5py.File`, opened for writing.

`paths`
: Entry path, e.g. “measurements/substrate/geometry”, or a list of entry paths, e.g. from `entries()`.

`overwrite`
: Replace attributes already present.

`**attrs`
: The attributes to write.

Raises

`ValueError`
: If an attribute is already present on one of the entries and `overwrite` is False; nothing is written then.

Example

```
>>> write_attrs("Virgo_EM01.h5", "measurements/substrate/geometry",
...             material="fused silica", mass=39.630, diameter=0.350,
...             thickness=0.200, wedge=1.07e-3)
>>> write_attrs("Virgo_EM01.h5", entries("Virgo_EM01.h5", tree="measurements"),
...             producer="LMA", document="VIR-0269A-15", date="2015-05")
```

`set_recommended` function

```
set_recommended(file: str | Path | File, surface: str, quantity: str, source: str,
    label: str | None = None, link: bool | None = None, producer: str | None = None,
    date: str | None = None, comment: str | None = None) -> str
```

Make an entry of another tree the recommended one for a quantity.

Creates or replaces `recommended/<surface>/<quantity>` (with `/<label>` if given). A map source becomes an HDF5 soft link to the source entry, so the map is stored once. Any other source is copied: its attributes except the campaign and provenance ones (`CAMPAIGN_ATTRS`) are written to the new entry together with `source`, the path of the origin, and the `producer`, `date` and `comment` of the choice.

Parameters

`file`
: Path or open `h5py.File`, opened for writing.

`surface`
: `s1`, `s2`, `substrate` or `ingot`.

`quantity`
: Quantity name, e.g. “roc” or “surface_height”.

`source`
: Path of the entry to recommend, e.g. “processed/s1/surface_height/350mm”.

`label`
: Label level of the recommended entry, e.g. “1064nm” for a wavelength-dependent quantity; omitted by default.

`link`
: Force a soft link (True) or a copy (False). By default a source holding a map is linked, anything else copied.

`producer`
: Who made the choice; written to a copied entry.

`date`
: ISO date of the choice; today by default, written to a copied entry.

`comment`
: Reason for the choice; written to a copied entry.

Returns

The path of the recommended entry.

Example

```
>>> set_recommended("Virgo_EM01.h5", "s1", "surface_height",
...                 "processed/s1/surface_height/350mm")
'recommended/s1/surface_height'
>>> set_recommended("Virgo_EM01.h5", "s1", "roc",
...                 "measurements/s1/roc/lma_2015-05",
...                 producer="A. Freise",
...                 comment="measured value from the producer's report")
'recommended/s1/roc'
```

`remove_entry` function

```
remove_entry(file: str | Path | File, path: str) -> None
```

Remove an entry from a file, or a group together with the entries in it.

Deletes the group or soft link at `path`, then the groups above it that are left without members and attributes. An entry that a recommended entry links to or was copied from is not removed, so that the recommended tree never refers to data that is gone: remove that recommended entry first, or point it elsewhere with `set_recommended()`.

Like the writers, the function leaves the history alone: call `append_history()` after the change. HDF5 does not release the space of deleted data, so the file does not shrink; after removing a substantial amount of data, generate a compact copy with the HDF5 command-line tool `h5repack old.h5 new.h5`.

Parameters

`file`
: Path or open `h5py.File`, opened for writing.

`path`
: Entry path, e.g. “measurements/s1/roc/lma_2015-05”, or a group path such as “measurements/s1/roc”.

Raises

`KeyError`
: If nothing exists at `path`.

`ValueError`
: If a recommended entry refers to the data at `path`.

Example

```
>>> remove_entry("Virgo_EM01.h5", "nominal/s1/transmission/1064nm")
>>> append_history("Virgo_EM01.h5", "A. Freise", "removed design transmission")
```

---

append_history function

```
append_history(file: str | Path | File, author: str, description: str,
    date: str | None = None) -> None
```

Record a modification: append to `file_history` and increment `revision`.

Call it after the writers that made a change, e.g. once per significant modification of an existing file. A file without history so far gets `revision = 1` and a one-line history.

Example

```
>>> append_history("Virgo_EM01.h5", "A. Freise",
...                "added 350mm extrapolated map")
```

4.2 Map utilities: mirror_maps.utils

Collection of general functions for processing map data. This module provides utilities for:

- loading map data from various file formats (HDF5, MetroPro, Finesse2);
- saving map data to HDF5 format;
- plotting and visualizing map data;
- handling NaN values and cropping maps;
- map preparation (removing piston, tilt, curvature);
- generating random maps for testing;
- various utility functions for map analysis;
- terminal output formatting (`color` class).

Anna Green and Andreas Freise, October 2023

Group	Functions
Loading and saving (4.2.1)	load_h5, save_h5, load_metropro, load_zeiss, load_finesse2, save_plot_pdf
Masking and cropping (4.2.2)	apply_circular_mask, crop_nans, crop_to_circle, crop_to_rect, extend_grid, recentre_map, max_radius
NaN handling (4.2.3)	check_map_NaNs, fill_NaN, fill_NaNs_ndimage, remove_NaN, create_amplitude_from_nans, map_coverage
Piston, tilt and curvature (4.2.4)	get_ptc_finesse, remove_ptc_finesse, apply_ptc_finesse, prepare_map, remove_mean, create_ptc_map
Statistics (4.2.5)	map_rms, compute_annulus_rms, radial_average_map
Plotting (4.2.6)	plot_map, plot_map_crossection, plot_radial_profile, plot_data_2D, plot_histogram, gwoptics_colormap, set_cbar_extent
Test maps and helpers (4.2.7)	create_random_map, create_random_rect_map, create_polar_grid, display_dict, is_odd, color
Deprecated (4.2.8)	fill_NaNs_mean, interpolate_NaNs, fill_circle, remove_curvature, remove_tilt_piston

4.2.1 Loading and saving

---

load_h5 function

```
load_h5(filename: str | Path) -> tuple[ndarray, ndarray, ndarray, dict[str, Any]]
```

Simple map loader for quick work, without the ET mirror data container.

Reads an HDF5 file with the root datasets ‘x’, ‘y’ and ‘A’ and returns the metadata stored as root attributes. This plain layout is enough for scratch files and intermediate results; files in the ET mirror data container format are read with `container.read_map()`.

Parameters

`filename`
: Path to the HDF5 file (with or without `.h5` extension).

Returns

Tuple `(x, y, Z, metadata)` where `x`, `y` are coordinate arrays, `Z` is the 2D surface height data, and `metadata` is a dict of attributes.

Raises

`FileNotFoundError`
: If the specified file does not exist.

Example

```
>>> x, y, Z, meta = load_h5("surface_map.h5")
```

---

save_h5 function

```
save_h5(x: ndarray, y: ndarray, Z: ndarray, filename: str,
    metadata: dict[str, Any] | None = None) -> None
```

Simple map writer for quick work, without the ET mirror data container.

Writes an HDF5 file with the root datasets ‘x’, ‘y’ and ‘A’, gzip compressed (level 9), and the metadata as root attributes. This plain layout is enough for scratch files and intermediate results; files meant for exchange are written in the ET mirror data container format with `container.create_file()` and `container.write_map()`.

Parameters

`x` 1D array of x-coordinates.

`y` 1D array of y-coordinates.

`Z` 2D array of surface height data with shape `(len(y), len(x))`.

`filename`
: Output filename (without `.h5` extension, added automatically).

`metadata`
: Optional dictionary of additional metadata to store.

Example

```
>>> save_h5(x, y, surface_data, "output_map",
...         metadata={"source": "measurement", "wavelength": 1064e-9})
```

---

load_metropro

function

```
load_metropro(filename: str | Path) -> tuple[ndarray, ndarray, ndarray]
```

Load map data from a MetroPro format file.

Uses Finesse's utility function to read MetroPro files (typically from Zygo interferometers) and converts the data to x, y, Z format.

Parameters

`filename`
Path to the MetroPro file.

Returns

Tuple `(x, y, Z)`: coordinate arrays and surface height data.

Raises

`FileNotFoundError`
If the specified file does not exist.

Example

```
>>> x, y, Z = load_metropro("measurement.dat")
```

---

load_zeiss

function

```
load_zeiss(filename: str | Path) -> tuple[ndarray, ndarray, ndarray]
```

Load map data from an ASCII export file provided for some mirrors obtained from Zeiss.

Reads XYZ ASCII files as exported by Zeiss interferometers (e.g. via the ompy toolchain). The file has a one-line header followed by rows of (x, y, z) in millimetres. Coordinates are converted to metres and the scattered points are reshaped onto a regular grid.

Grid cells that have no corresponding data point in the file are set to NaN in the output. Note that the returned coordinates preserve the original Zeiss positions (not centred on zero).

Parameters

`filename`
Path to the Zeiss ASCII file (`.asc`).

Returns

Tuple `(x, y, Z)` where `x`, `y` are 1D coordinate arrays [m] and `Z` is a 2D surface height array [m] with NaN outside the measured region.

Raises

`FileNotFoundError`
If the specified file does not exist.

Example

```
>>> x, y, Z = load_zeiss("TM12_konkav_Passe_240917.asc")
>>> Z_nm = Z * 1e9  # convert to nanometres
```

---

load_finesse2 function

```
load_finesse2(filename: str | Path) -> tuple[ndarray, ndarray, ndarray]
```

Load map data from a Finesse 2 format file.

Reads ASCII map files in the format used by Finesse 2 / PyKat, which include a header with metadata followed by the map data.

Parameters

`filename`
Path to the Finesse 2 map file.

Returns

Tuple `(x, y, Z)`: coordinate arrays and scaled surface height data.

Notes

File format expects header lines with name, type, size, center, step, and scaling factor. Data starts at line 10.

Raises

`FileNotFoundError`
If the specified file does not exist.

Example

```
>>> x, y, Z = load_finesse2("mirror_map.txt")
```

---

save_plot_pdf function

```
save_plot_pdf(fig: Figure, filename: str) -> None
```

Save a matplotlib figure to a compact PDF file.

Parameters

`fig`
Matplotlib figure object to save.

`filename`
Output filename (should include `.pdf` extension).

Example

```
>>> fig, ax, _ = plot_map(x, y, Z)
```

```
>>> save_plot_pdf(fig, "surface_plot.pdf")
```

4.2.2 Masking, cropping and grids

---

apply_circular_mask

function

```
apply_circular_mask(x: ndarray, y: ndarray, A: ndarray, Rmax: float | None = None,
    Rmin: float = 0.0, fill_value: float | None = None, debug: bool = False,
    force_square: bool = False) -> tuple[ndarray, ndarray, ndarray]
```

Apply a circular mask to map data.

Sets data points outside `Rmax` and/or inside `Rmin` to a fill value (default NaN). Can create circular apertures, central obscurations, or annular regions.

Parameters

`x` 1D array of x-coordinates.

`y` 1D array of y-coordinates.

`A` 2D array of map data with shape `(len(y), len(x))`.

`Rmax`
Outer radius. Points with $R > R_{\max}$ are filled. If None, no outer mask.

`Rmin`
Inner radius. Points with $R < R_{\min}$ are filled. Default 0 (no inner mask).

`fill_value`
Value to use in masked regions. Default None (NaN).

`debug`
If True, print diagnostic information. Default False.

`force_square`
If True, adjust coordinates for exactly square pixel array.

Returns

Tuple `(x, y, A)` with mask applied. Always returns a tuple for consistency.

Example

```
>>> # Mask outside r=0.05 (circular aperture)
>>> x, y, A_masked = apply_circular_mask(x, y, surface, Rmax=0.05)

>>> # Mask inside r=0.02 (central obscuration)
>>> x, y, A_masked = apply_circular_mask(x, y, surface, Rmin=0.02)

>>> # Create annular region 0.02 < r < 0.05
>>> x, y, A_masked = apply_circular_mask(x, y, surface, Rmax=0.05, Rmin=0.02)
```

crop_nans function

```
crop_nans(x: ndarray, y: ndarray, A: ndarray) -> tuple[ndarray, ndarray, ndarray]
```

Remove rows and columns that are entirely NaN.

Parameters

`x` 1D array of x-coordinates.

`y` 1D array of y-coordinates.

`A` 2D array of map data with shape `(len(y), len(x))`.

Returns

Tuple `(x, y, A)` with NaN-only rows/columns removed.

Example

```
>>> x_crop, y_crop, A_crop = crop_nans_map(x, y, surface_data)
```

crop_to_circle function

```
crop_to_circle(x: ndarray, y: ndarray, A: ndarray, crop_radius: float,
    replace_nan: bool = True) -> tuple[ndarray, ndarray, ndarray]
```

Crop map arrays to tightly fit around a circular aperture.

Applies a circular mask and then crops to the smallest bounding box that contains all valid (non-NaN) data.

Parameters

`x` 1D array of x-coordinates.

`y` 1D array of y-coordinates.

`A` 2D array of map data with shape `(len(y), len(x))`.

`crop_radius`
: Radius of circular aperture in same units as x, y.

`replace_nan`
: If True (default), replace NaN values with zeros.

Returns

Tuple `(x, y, A)` cropped to fit the circular region.

See also

`circular_mask`: Apply mask without cropping.

Example

```
>>> x_crop, y_crop, A_crop = crop_to_circle(x, y, A, crop_radius=0.05)
```

crop_to_rect

function

```
crop_to_rect(x: ndarray, y: ndarray, A: ndarray, cx: float, cy: float)
    -> tuple[ndarray, ndarray, ndarray]
```

Extract a rectangular region from map data.

Crops the map to a centered rectangle of specified dimensions, ensuring an even number of points in each dimension.

Parameters

`x` 1D array of x-coordinates.

`y` 1D array of y-coordinates.

`A` 2D array of map data with shape `(len(y), len(x))`.

`cx`
Desired width of cropped region in same units as x.

`cy`
Desired height of cropped region in same units as y.

Returns

Tuple `(x, y, A)` cropped to the specified dimensions.

Example

```
>>> x_crop, y_crop, A_crop = rect_crop(x, y, surface, 0.1, 0.1)
```

extend_grid

function

```
extend_grid(x: ndarray, y: ndarray, Z: ndarray, new_radius: float,
    fill_value: float = nan) -> tuple[ndarray, ndarray, ndarray]
```

Extend coordinate arrays to a larger radius, filling new regions.

Useful for extending measured mirror data to the full physical mirror size before Zernike reconstruction. The grid spacing is preserved from the original data.

Parameters

`x` 1D array of x-coordinates (must be centered, i.e., symmetric).

`y` 1D array of y-coordinates (must be centered, i.e., symmetric).

`Z` 2D data array with shape `(len(y), len(x))`.

`new_radius`
Target radius for the extended grid (same units as x, y).

`fill_value`
Value to fill in extended regions. Default `np.nan`.

Returns

Tuple `(x_ext, y_ext, Z_ext)`: extended coordinate and data arrays.

Raises

`ValueError`
If `new_radius` is smaller than existing data extent.

`ValueError`
If x or y arrays are not centered (symmetric about 0).

Example

```
>>> # Extend 0.15m data to 0.175m mirror radius
>>> x_ext, y_ext, Z_ext = extend_grid(x, y, Z, new_radius=0.175)
```

`recentre_map` function

```
recentre_map(x: ndarray, y: ndarray, offset: list[float] | tuple[float, float] | None = None,
    debug: bool = False) -> tuple[ndarray, ndarray]
```

Shift map coordinates to a new center.

Parameters

`x` 1D array of x-coordinates.

`y` 1D array of y-coordinates.

`offset`
Tuple `(x0, y0)` for the offset. If None, uses center of gravity.

`debug`
If True, print the offset being applied. Default False.

Returns

Tuple `(x - x0, y - y0)`: shifted coordinate arrays.

Example

```
>>> x_centered, y_centered = recentre_map(x, y, offset=(0.001, -0.002))
```

`max_radius` function

```
max_radius(x: ndarray, y: ndarray) -> float
```

Calculate the maximal inscribed disk radius in a 2D grid.

Returns the largest radius for which a centered circle fits entirely within the rectangular grid.

Parameters

`x` 1D array of x-coordinates.

`y` 1D array of y-coordinates.

Returns

The maximum radius in same units as x, y.

Example

```
>>> Rmax = get_maximal_radius(x, y)
```

4.2.3 NaN handling

---

check_map_NaNs function

```
check_map_NaNs(x: ndarray, y: ndarray, Z: ndarray, plot: bool = False,
    title_prefix: str = '') -> int
```

Check and visualize NaN values in map data.

Prints summary statistics about NaN locations and optionally displays side-by-side plots of the map data and NaN locations.

Parameters

`x` 1D array of x-coordinates.

`y` 1D array of y-coordinates.

`Z` 2D array of map data, may contain NaNs.

`plot`
If True, display visualization of map and NaN locations.

`title_prefix`
Prefix for plot titles.

Returns

Number of NaN values found (`int`).

Example

```
>>> n_nans = check_map_NaNs(x, y, surface_data, plot=True)
```

---

fill_NaN function

```
fill_NaN(x: ndarray, y: ndarray, A: ndarray, margin: int = 3)
    -> tuple[ndarray, ndarray, ndarray]
```

Fill the NaN pixels inside the data by linear interpolation.

Every NaN pixel inside the convex hull of the valid pixels is filled: the gaps in the interior and the notches along the edge of the data. The NaN region outside the hull, beyond a circular aperture for example, is left as it is. The interpolation is evaluated on a window of `margin` pixels around each gap instead of the whole map, which is much faster and uses less memory. The window is enlarged where a gap needs more distant valid pixels. Only measured pixels are used as support, never pixels filled before.

Parameters

`x` 1D array of x-coordinates.

`y` 1D array of y-coordinates.

`A` 2D array of map data, may contain NaNs.

`margin`
Number of pixels added on every side of a gap's bounding box to form the interpolation window. Default 3.

Returns

Tuple `(x, y, A_filled)` with the NaN values inside the data interpolated.

Notes

Inside a gap the linear interpolation depends on the diagonal the triangulation picks for each pixel square. That choice is arbitrary on a square grid, so a few filled values can differ from those of a triangulation of the whole map, by up to the local pixel-to-pixel height difference.

Example

```
>>> x, y, A_filled = fill_NaN(x, y, surface_with_gaps)
```

`fill_NaNs_ndimage` function

```
fill_NaNs_ndimage(A: ndarray) -> ndarray
```

Fast NaN filling using `scipy.ndimage` distance transform (nearest-neighbor).

Note: this fills *all* NaNs including boundary.

Parameters

`A` 2D array potentially containing NaN values.

Returns

Copy of the array with all NaN values replaced by nearest valid neighbour.

`remove_NaN` function

```
remove_NaN(A: ndarray) -> ndarray
```

Replace all NaN values with zero.

Parameters

`A` NumPy array potentially containing NaN values.

Returns

Copy of array with NaNs replaced by 0.0.

Example

```
>>> A_clean = remove_NaN(surface_data)
```

`create_amplitude_from_nans` function

```
create_amplitude_from_nans(Z: ndarray) -> ndarray
```

Create amplitude map: 1 where data is valid, 0 where NaN.

Parameters

`Z` 2D array potentially containing NaN values.

Returns

Binary amplitude map (1.0 for valid data, 0.0 for NaN).

Example

```
>>> amp = create_amplitude_from_nans(surface_data)
```

`map_coverage` function

```
map_coverage(x: ndarray, y: ndarray, Z: ndarray, radius: float | None = None) -> float
```

Calculate the fraction of valid (non-NaN) data points in a map.

Parameters

`x` 1D array of x-coordinates.

`y` 1D array of y-coordinates.

`Z` 2D array of map data with shape `(len(y), len(x))`.

`radius`
If provided, only consider points within this radius.

Returns

Fraction of valid data points (0.0 to 1.0).

Example

```
>>> coverage = map_coverage(x, y, surface)
>>> print(f"Data coverage: {coverage * 100:.1f}%")
```

4.2.4 Piston, tilt and curvature

`get_ptc_finesse` function

```
get_ptc_finesse(x: ndarray, y: ndarray, _A: ndarray, spot_size_weight: float,
    debug: bool = False, map_scale: float = 1e-09)
    -> tuple[float, float, float, float, float]
```

Returns piston, tilt, and curvature using Finesse's overlap methods.

NaN values in the input are replaced with zero for the overlap computation.

Parameters

`x` 1D array of x-coordinates.

`y` 1D array of y-coordinates.

`_A`
2D array of map data with shape `(len(y), len(x))`. May contain NaN.

`spot_size_weight`
Gaussian beam spot size for weighting the overlap.

`debug`
If True, print computed values. Default False.

`map_scale`
Scale factor for converting map units to meters. Default 1e-9.

Returns

Tuple `(piston, yaw, pitch, x RoC, y RoC)`.

Example

```
>>> piston, yaw, pitch, Rcx, Rcy = get_ptc_finesse(x, y, surface, 0.05)
```

`remove_ptc_finesse` function

```
remove_ptc_finesse(x: ndarray, y: ndarray, _A: ndarray, spot_size_weight: float,
    piston: bool = True, tilt: bool = True, roc: bool = True, debug: bool = False)
    -> tuple[ndarray, ndarray, ndarray]
```

Remove piston, tilt, and/or curvature using Finesse's overlap methods.

Uses Finesse's spot-size-weighted overlap integrals to compute and remove low-order aberrations from map data. NaN values in the input are preserved in the output (correction is only applied to valid data).

Parameters

`x` 1D array of x-coordinates.

`y` 1D array of y-coordinates.

`_A`
2D array of map data with shape `(len(y), len(x))`. May contain NaN.

`spot_size_weight`
Gaussian beam spot size for weighting the overlap.

`piston`
Whether to remove piston (constant offset). Default True.

`tilt`
Whether to remove tilt (linear terms). Default True.

`roc`
Whether to remove curvature (quadratic terms). Default True.

`debug`
If True, print removed values and show plot. Default False.

Returns

Tuple `(x, y, corrected_map)` with aberrations removed. NaN preserved.

Example

```
>>> x, y, A_flat = remove_ptc_finesse(x, y, surface, spot_size_weight=0.01,
...                                   piston=True, tilt=True, roc=True)
```

apply_ptc_finesse

function

```
apply_ptc_finesse(x: ndarray, y: ndarray, Z: ndarray, beam_radius: float,
    n_iter: int = 4, piston: bool = True, tilt: bool = True, roc: bool = True) -> ndarray
```

Iterated Finesse PTC removal.

Applies `remove_ptc_finesse` repeatedly so the spot-size-weighted overlap integrals converge. Typically only the piston changes appreciably after the first pass; four iterations is usually more than enough (for example, on the extrapolated Virgo source maps the corrections from the third iteration on are below 10^{-7}).

Parameters

`x` 1D array of x-coordinates.

`y` 1D array of y-coordinates.

`Z` 2D array of map data with shape `(len(y), len(x))`. May contain NaN.

`beam_radius`
: Gaussian beam spot size for the overlap weighting.

`n_iter`
: Number of PTC removal iterations. Default 4.

`piston`
: Remove piston. Default True.

`tilt`
: Remove tilt. Default True.

`roc`
: Remove curvature. Default True.

Returns

Map data with PTC removed (same shape as `Z`, NaN preserved).

prepare_map

function

```
prepare_map(x: ndarray, y: ndarray, Z: ndarray, spotsize: float, Rmax: float,
    debug: bool = False) -> ndarray
```

Prepare map data by removing aberrations and applying circular mask.

Removes piston, tilt, and curvature using Gaussian-weighted overlap integrals, then applies a circular aperture mask. *TODO: maybe change to using ptc four times.*

Parameters

`x` 1D array of x-coordinates.

`y` 1D array of y-coordinates.

`Z` 2D array of map data with shape `(len(y), len(x))`.

`spotsize`
: Gaussian beam spot size for weighting aberration removal.

`Rmax`
: Radius of the circular aperture mask.

`debug`
If True, print diagnostic information. Default False.

Returns

Prepared map with aberrations removed and circular mask applied.

See also

`remove_ptc_finesse`: for more control over which aberrations to remove.

Example

```
>>> Z_prepared = prepare_map(x, y, surface_data, spotsize=0.01, Rmax=0.05)
```

---

remove_mean

function

```
remove_mean(x: ndarray, y: ndarray, A: ndarray, R: float | None = None,
    debug: bool = False) -> ndarray
```

Remove the mean value from map data.

NaN values are ignored when computing the mean and preserved in the output.

Parameters

`x` 1D array of x-coordinates.

`y` 1D array of y-coordinates.

`A` 2D array of map data with shape `(len(y), len(x))`. May contain NaN.

`R` If provided, compute and subtract mean only within this radius. Points outside R are unchanged. Default None (use all data).

`debug`
If True, print the computed mean. Default False.

Returns

Map data with mean subtracted. NaN values are preserved.

Example

```
>>> A_zero_mean = remove_mean(x, y, surface, R=0.05)
```

---

create_ptc_map

function

```
create_ptc_map(x: ndarray, y: ndarray, piston: float = 0.0, yaw: float = 0.0,
    pitch: float = 0.0, a: float = 0.0, b: float = 0.0, debug: bool = False) -> ndarray
```

Create a surface map with piston, tilt, and curvature aberrations.

The aberration terms follow the same formulation as in `remove_ptc_finesse()`:

- **Piston**: constant offset (nm);
- **Tilt**: $\text{yaw} \cdot X + \text{pitch} \cdot Y$ where yaw, pitch are in nm/m;
- **Curvature**: $aX^2 + bY^2$ where a, b are in nm/m^2.

Parameters

`x` 1D array of x-coordinates in meters.

`y` 1D array of y-coordinates in meters.

`piston`
Piston offset in nm. Default 0.0.

`yaw`
Tilt in x-direction in nm/m. Default 0.0.

`pitch`
Tilt in y-direction in nm/m. Default 0.0.

`a` X-curvature coefficient in nm/m^2. Default 0.0.

`b` Y-curvature coefficient in nm/m^2. Default 0.0.

`debug`
If True, print aberration values. Default False.

Returns

2D array of aberrations in nm with shape `(len(y), len(x))`.

Example

```
>>> aberrations = create_ptc_map(x, y, piston=10, yaw=100, a=1e6)
```

4.2.5 Statistics

`map_rms` function

```
map_rms(x: ndarray, y: ndarray, A: ndarray, Rmax: float | None = None,
    mask_only: bool = True) -> float
```

Calculate the RMS (root mean square) of map data.

Computes the RMS value, optionally within a circular aperture. NaN values are excluded from the calculation.

Parameters

`x` 1D array of x-coordinates.

`y` 1D array of y-coordinates.

`A` 2D array of map data with shape `(len(y), len(x))`.

`Rmax`
If provided, compute RMS only within this radius. Default None.

`mask_only`
If True (default), only points inside the aperture are used. If False, points outside are set to zero (legacy behavior).

Returns

The RMS value $\sqrt{\text{mean}(A^2)}$.

Example

```
>>> rms = map_RMS(x, y, surface_data, Rmax=0.05)
```

---

compute_annulus_rms

function

```
compute_annulus_rms(x: ndarray, y: ndarray, Z: ndarray, beam_radius: float,
    edges: ndarray | list[float]) -> ndarray
```

RMS of map data in concentric annuli scaled by beam radius.

Annulus i covers radii $[\text{edges}[i] \cdot w,\ \text{edges}[i+1] \cdot w)$, with w the beam radius. NaN values are excluded.

Parameters

`x` 1D array of x-coordinates.

`y` 1D array of y-coordinates.

`Z` 2D array of map data with shape `(len(y), len(x))`. May contain NaN.

`beam_radius`
: Beam radius w used to scale the edges.

`edges`
: Annulus edges in units of beam radius (e.g. `[0, 0.5, 1, 1.5]`).

Returns

RMS in each annulus, length `len(edges) - 1`. Empty annuli return 0.0.

---

radial_average_map

function

```
radial_average_map(x: ndarray, y: ndarray, Z: ndarray, N_bins: int = 100,
    debug: bool = False) -> tuple[ndarray, ndarray, ndarray]
```

Compute the radially averaged profile of 2D map data.

Bins data by radial distance from center and computes mean/std in each bin. NaN values are ignored in the computation.

Parameters

`x` 1D array of x-coordinates.

`y` 1D array of y-coordinates.

`Z` 2D array of map data with shape `(len(y), len(x))`. May contain NaN.

`N_bins`
: Number of radial bins. Default 100.

`debug`
: If True, print number of output points. Default False.

Returns

Tuple `(r, avg, std)`: radial bin centers, means, and standard deviations.

Example

```
>>> r, avg_profile, std_profile = radial_average_map(x, y, surface)
```

4.2.6 Plotting

---

plot_map function

```
plot_map(x: ndarray, y: ndarray, Z: ndarray, figsize: tuple[int, int] = (5, 5),
    Zmax: float | tuple[float, float] | None = None, center: float | None = 0.0,
    clip_radius: float | None = None, Z_label: str = 'Z [a.u.]', title: str | None = None,
    colorbar: bool = True, colormap: Any | None = None, location: str | None = None,
    orientation: str | None = None, ax: Axes | None = None, tight_layout: bool = True,
    barshrink: float = 0.5, barpad: float = 0.1, hatches: bool = False,
    hatch: str = '//', showaxes: bool = True) -> tuple[Figure, Axes, Any | None]
```

Plot 2D map data with customizable appearance.

Creates a color mesh plot of map data with options for clipping, custom colormaps, hatching patterns, and more.

Parameters

`x` 1D array of x-coordinates.

`y` 1D array of y-coordinates.

`Z` 2D array of map data with shape `(len(y), len(x))`.

`figsize`
: Figure size tuple (width, height). Default (5, 5).

`Zmax`
: Colour scale limits: None spans the data, a float gives symmetric limits ±`Zmax` about `center`, a tuple (lower, upper) gives explicit limits. With a centre set, tuple limits are widened to be symmetric about it; the colour bar arrows show where the data exceed the limits.

`center`
: Value at the centre of the colour scale. The default 0.0 centres the scale on zero, for residual maps with the default diverging colormap. None spans the data or the given limits without a centre, for maps of one sign such as a surface height with its curvature, best with a sequential colormap.

`clip_radius`
: If provided, apply circular mask and limit plot extent.

`Z_label`
: Label for the colorbar. Default 'Z [a.u.]'.

`title`
: Optional title for the plot.

`colorbar`
: Whether to show colorbar. Default True.

`colormap`
: Matplotlib colormap. Default uses `cmap_r` from `seasons_s`.

`location`
: Colorbar location ('left', 'right', 'top', 'bottom').

`orientation`
Colorbar orientation ('horizontal' or 'vertical').

`ax`
Existing axes to plot on. If None, creates new figure.

`tight_layout`
Whether to apply `tight_layout`. Default True.

`barshrink`
Colorbar shrink factor. Default 0.5.

`barpad`
Colorbar padding. Default 0.1.

`hatches`
Whether to add hatch pattern for near-zero values. Default False.

`hatch`
Hatch pattern string. Default '//'.

`showaxes`
Whether to show axis labels. Default True.

Returns

Tuple `(fig, ax, clb)`: Figure, axes, and colorbar objects.

Example

```
>>> fig, ax, clb = plot_map(x, y, surface, Zmax=1e-9, clip_radius=0.05)
>>> fig, ax, clb = plot_map(x, y, height, center=None, colormap="viridis")
```

`plot_map_crossection` function

```
plot_map_crossection(x: ndarray, y: ndarray, Z: ndarray, clip_radius: float | None = None,
    row_idx: int | None = None, col_idx: int | None = None, Z_label: str = 'Z [a.u.]',
    title: str | None = None, ax: Axes | None = None) -> Axes
```

Plot horizontal and vertical cross-sections through map data.

Parameters

`x` 1D array of x-coordinates.

`y` 1D array of y-coordinates.

`Z` 2D array of map data with shape `(len(y), len(x))`.

`clip_radius`
If provided, apply circular aperture mask before plotting.

`row_idx`
Row index for horizontal cross-section. Default is middle row.

`col_idx`
Column index for vertical cross-section. Default is middle column.

`Z_label`
Label for y-axis. Default 'Z [a.u.]'.

`title`
Optional title for the plot.

`ax`
: Existing axes to plot on. If None, creates new figure.

Returns

The axes object (`matplotlib.axes.Axes`).

Example

```
>>> ax = plot_map_crossection(x, y, surface, clip_radius=0.05)
```

`plot_radial_profile` function

```
plot_radial_profile(x: ndarray, y: ndarray, Z: ndarray, ax: Axes | None = None,
    label: str | None = None, show_std: bool = True, N_bins: int = 100, **kwargs) -> Axes
```

Plot radial average profile of 2D map data.

Computes the radial average using `radial_average_map()` and plots the result with optional standard deviation band.

Parameters

`x` 1D array of x-coordinates.

`y` 1D array of y-coordinates.

`Z` 2D array of map data with shape `(len(y), len(x))`.

`ax`
: Existing axes to plot on. If None, creates new figure.

`label`
: Legend label for the plot.

`show_std`
: If True, show standard deviation as shaded band. Default True.

`N_bins`
: Number of radial bins. Default 100.

`**kwargs`
: Additional arguments passed to `ax.plot()`.

Returns

The axes object (`matplotlib.axes.Axes`).

Example

```
>>> fig, ax = plt.subplots()
>>> plot_radial_profile(x, y, original, ax=ax, label='Original')
>>> plot_radial_profile(x, y, reconst, ax=ax, label='Fit', show_std=False)
```

plot_data_2D function

```
plot_data_2D(x: ndarray, y: ndarray, A: ndarray, ax: Axes | None = None,
    figsize: tuple[int, int] = (5, 4), cmap: Any | None = None) -> tuple[Figure, Axes]
```

Create a simple 2D color plot of array data.

Parameters

`x` 1D array of x-coordinates.

`y` 1D array of y-coordinates.

`A` 2D array of data with shape `(len(y), len(x))`.

`ax`
Existing axes to plot on. If None, creates new figure.

`figsize`
Figure size tuple. Default (5, 4).

`cmap`
Matplotlib colormap. If None, uses `gwoptics_colormap`.

Returns

Tuple `(fig, ax)`: Figure and axes objects.

Example

```
>>> fig, ax = plot_data_2D(x, y, intensity_map)
```

plot_histogram function

```
plot_histogram(x: ndarray, y: ndarray, xlabel: str = 'x', ylabel: str = 'y',
    plottype: str = 'bar', title: str | None = None, ax: Axes | None = None,
    figsize: tuple[int, int] | None = None, logy: bool = False, grid: bool = False,
    tight_layout: bool = True, **kwargs) -> Axes
```

Create a histogram or scatter plot of data.

Parameters

`x` Array of x-values (categories or positions).

`y` Array of y-values (heights or counts).

`xlabel`
Label for x-axis. Default ‘x’.

`ylabel`
Label for y-axis. Default ‘y’.

`plottype`
Type of plot — “bar” or “scatter”. Default “bar”.

`title`
Optional title for the plot.

`ax`
Existing axes to plot on. If None, creates new figure.

`figsize`
Figure size tuple. If None, auto-computed from data length.

`logy`
Whether to use logarithmic y-scale. Default False.

`grid`
Whether to show grid lines. Default False.

`tight_layout`
Whether to apply `tight_layout`. Default True.

`**kwargs`
Additional keyword arguments passed to `bar()` or `scatter()`.

Returns

The axes object (`matplotlib.axes.Axes`).

Example

```
>>> ax = plot_histogram(bins, counts, xlabel='Value', logy=True)
```

---

gwoptics_colormap

function

```
gwoptics_colormap(Npoints: int = 128) -> ListedColormap
```

Create the classic GWOptics/SimTools colormap.

Recreates the colormap used in older SimTools and gnuplot files for gravitational wave optics simulations.

Parameters

`Npoints`
Number of color points in the colormap. Default 128.

Returns

The colormap object (`matplotlib.colors.ListedColormap`).

Example

```
>>> cmap = gwoptics_colormap()
>>> plt.imshow(data, cmap=cmap)
```

---

set_cbar_extent

function

```
set_cbar_extent(Z: ndarray, Zmax: tuple[float, float] | list[float]) -> str
```

Determine the colorbar extension arrows based on data range.

Parameters

`Z` 2D array of data values (may contain NaNs).

`Zmax`
Colorbar limits as (min, max) tuple or list.

Returns

One of 'both', 'max', 'min', or 'neither'.

Example

```
>>> extent = set_cbar_extent(surface_data, [-1e-9, 1e-9])
```

4.2.7 Test maps and helpers

---

create_random_map function

```
create_random_map(x: ndarray, y: ndarray) -> ndarray
```

Generate a random map with zero mean.

Parameters

`x` 1D array of x-coordinates (defines output columns).

`y` 1D array of y-coordinates (defines output rows).

Returns

2D array of random values with zero mean, shape `(len(y), len(x))`.

Example

```
>>> A = random_map(x, y)
```

---

create_random_rect_map function

```
create_random_rect_map(xwidth: float, ywidth: float, xpoints: int, ypoints: int)
    -> tuple[ndarray, ndarray, ndarray]
```

Generate a random rectangular map with specified dimensions.

Parameters

`xwidth`
: Total width of the map in x-direction.

`ywidth`
: Total width of the map in y-direction.

`xpoints`
: Number of points in x-direction.

`ypoints`
: Number of points in y-direction.

Returns

Tuple `(x, y, A)`: coordinate arrays and random map with zero mean.

Example

```
>>> x, y, A = random_rect_map(0.1, 0.1, 100, 100)
```

create_polar_grid

function

```
create_polar_grid(A: ndarray, centre_offset: tuple[float, float] = (0, 0),
    R0: float | None = None, debug: bool = False) -> tuple[ndarray, ndarray]
```

Generate normalized polar coordinates for a 2D array.

Parameters

`A` 2D numpy array defining the grid shape.

`centre_offset`
Tuple (x, y) offset of center in pixels. Default (0, 0).

`R0`
Normalization radius in pixels. If None, auto-computed. Default None.

`debug`
If True, print detailed coordinate information. Default False.

Returns

Tuple `(rho, phi)`: normalized radial and angular coordinates.

Example

```
>>> rho, phi = polar_grid(surface_data)
```

display_dict

function

```
display_dict(my_dict: dict[str, Any], max_key_width: int = 40,
    max_value_width: int = 40) -> None
```

Print a dictionary as a formatted table.

Parameters

`my_dict`
Dictionary to display.

`max_key_width`
Maximum width for key column. Default 40.

`max_value_width`
Maximum width for value column. Default 40.

Example

```
>>> display_dict({"source": "measurement", "date": "2024-01-15"})
```

is_odd

function

```
is_odd(num: int) -> bool
```

Check if a number is odd.

---

color class

```
class color
```

Bases: object

ANSI escape codes for colored terminal output.

```
BLUE = '\x1b[94m'

BOLD = '\x1b[1m'

CYAN = '\x1b[96m'

DARKCYAN = '\x1b[36m'

END = '\x1b[0m'

GREEN = '\x1b[92m'

PURPLE = '\x1b[95m'

RED = '\x1b[91m'

UNDERLINE = '\x1b[4m'

YELLOW = '\x1b[93m'
```

Example

```
>>> print(f"{color.RED}Error:{color.END} Something went wrong")
```

4.2.8 Deprecated functions

---

fill_NaNs_mean function

```
fill_NaNs_mean(A: ndarray) -> ndarray
```

Replace NaN values with the mean of neighboring pixels.

For each NaN, computes the average of valid 4-connected neighbors.

Deprecated. This function fails to fill NaN clusters. Use `fill_NaNs()` for better NaN filling that can preserve the outer NaN border.

Parameters

`A` 2D array potentially containing NaN values.

Returns

Copy of the array with NaN values replaced.

Warning

May fail if multiple adjacent pixels are NaN.

Example

```
>>> A_filled = replace_NaNs_mean(surface_data)
```

---

interpolate_NaNs function

```
interpolate_NaNs(A: ndarray) -> ndarray
```

Replace NaN values using nearest-neighbor interpolation.

Deprecated. This function is slow for large arrays. Use `fill_NaNs_ndimage()` for faster nearest-neighbor interpolation.

Parameters

`A` 2D array potentially containing NaN values.

Returns

Array with NaN values replaced by nearest neighbor values.

Example

```
>>> A_filled = interpolate_NaNs(surface_data)
```

---

fill_circle function

```
fill_circle(x: ndarray, y: ndarray, A: ndarray, Rmax: float | None,
    fill_value: float | None = None, debug: bool = False)
    -> tuple[ndarray, ndarray, ndarray]
```

Fill the inside of a circular region with a specified value.

Deprecated. Use `apply_circular_mask(x, y, A, Rmin=radius)` instead.

Parameters

`x` 1D array of x-coordinates.

`y` 1D array of y-coordinates.

`A` 2D array of map data with shape `(len(y), len(x))`.

`Rmax`
Radius of the circular region to fill. Points with $r < R_{\max}$ are filled.

`fill_value`
Value to use inside the circle. Default None (NaN).

`debug`
If True, print diagnostic information. Default False.

Returns

Tuple `(x, y, A)` with inner circle filled.

---

remove_curvature

function

```
remove_curvature(x: ndarray, y: ndarray, A: ndarray, spot_size_weight: float,
    debug: bool = False) -> ndarray
```

Remove curvature (radius of curvature) from map data.

Deprecated. Use `remove_ptc_finesse()` with `piston=False, tilt=False, roc=True` instead.

Parameters

`x` 1D array of x-coordinates.

`y` 1D array of y-coordinates.

`A` 2D array of map data with shape `(len(y), len(x))`.

`spot_size_weight`
: Gaussian beam spot size for weighting.

`debug`
: If True, print the removed radii of curvature. Default False.

Returns

Map data with curvature removed.

Example

```
>>> A_flat = remove_curvature(x, y, surface, spot_size_weight=0.01)
```

---

remove_tilt_piston

function

```
remove_tilt_piston(x: ndarray, y: ndarray, A: ndarray, spot_size_weight: float,
    debug: bool = False) -> ndarray
```

Remove tilt and piston from map data using Finesse overlap methods.

Deprecated. Use `remove_ptc_finesse()` with `piston=True, tilt=True, roc=False` instead.

Parameters

`x` 1D array of x-coordinates.

`y` 1D array of y-coordinates.

`A` 2D array of map data with shape `(len(y), len(x))`.

`spot_size_weight`
: Gaussian beam spot size for weighting.

`debug`
: If True, print removed values. Default False.

Returns

Map data with piston and tilt removed.

Example

```
>>> A_corrected = remove_tilt_piston(x, y, surface, spot_size_weight=0.01)
```

4.3 Zernike polynomials: `mirror_maps.zernike`

Zernike polynomial analysis and map generation. Tools for generating Zernike polynomials and using them to analyze and synthesize circular mirror map surfaces. Provides functions for decomposing surface maps into Zernike coefficients and generating random maps with prescribed Zernike statistics.

Key features

- Zernike polynomial computation with multiple normalization options.
- Efficient recurrence relations for high-order polynomials.
- Bidirectional index conversion ($(n, m) \leftrightarrow i$ and Noll j).
- Surface decomposition into Zernike coefficients.
- Random map generation weighted by layer coefficients.
- Visualization tools for Zernike pyramids and coefficient histograms.

Main functions

Function	Purpose
Core polynomials (Section 4.3.1)	
`Znm`	Complete Zernike polynomial $Z_n^m(\rho, \phi)$
`Rnm`	Radial polynomial $R_n^m(\rho)$
`Rnm_recurrence`	Radial polynomial by Chong–Kintner recurrence
`Anm`	Azimuthal function (cos/sin selection)
Index conversion (Section 4.3.2)	
`get_i, get_nm`	Convert between (n, m) and sequential i index
`get_j_Noll, get_nm_Noll`	Convert between (n, m) and Noll j index
`list_nm, Noll_order_basis`	Index arrays and Noll reordering
`get_Znm_name`	Human-readable names
Map analysis (Section 4.3.3)	
`zernike_basis`	Generate basis cuboid of Zernike polynomials
`get_cnm`	Decompose map into Zernike coefficients
`get_cnm_robust`	Least-squares decomposition for partial coverage
`fit_zernike`	Fit a map on its physical grid in one call, with n and m
`make_zernike_map`	Synthesize map from coefficients
`rms_from_cnm`	Compute RMS from coefficients
`remove_ptc_zernike`	Remove piston, tilt, curvature by Zernike fitting
`zero_cnm_terms`	Set selected coefficients to zero
Random generation (Section 4.3.4)	
`get_Bn`	Compute layer coefficients from c_{nm}
`random_cnm_flat`	Uniform random coefficients
`random_cnm_Bn`	Random coefficients weighted by B_n (quadrature)
`random_cnm_simtools`	Random coefficients weighted by B_n (absolute sum)
Visualization (Section 4.3.5)	
`plot_Znm`	Plot single Zernike polynomial
`plot_zernike_pyramid`	Plot pyramid of polynomials up to order n
`plot_basis`	Plot all terms of a basis
`make_pyramid_figure`	Figure with pyramidal subplot arrangement
`plot_cnm_histogram`	Histogram of Zernike coefficients
`plot_Bn_histogram`	Histogram of layer coefficients

Dependencies

Requires: `numpy`, `matplotlib`, `math`, `warnings`. Uses: `utils.fill_NaNs_mean`, `utils.create_polar_grid`, `utils.plot_histogram`, `utils.plot_map`.

Authors

Anna Green, Andreas Freise, January 2026

References

[1] C. Bond et al., "Interferometer Techniques for GW Detection", Living Reviews in Relativity, 2017.

[2] L. Tao et al., PRD 102, 122002 (2020).

[3] R. Noll, "Zernike polynomials and atmospheric turbulence", JOSA 66(3), 1976.

[4] Poppy documentation: `https://poppy-optics.readthedocs.io/`

4.3.1 Core polynomials

`Znm` function

```
Znm(n: int, m: int, rho: ndarray, phi: ndarray, normalisation: str = None, n0: int = 25,
    outside: float = 0, _outside_mask: ndarray = None, _rho_powers: dict = None,
    _azimuthal: dict = None) -> ndarray
```

Compute the Zernike polynomial Z_n^m in polar coordinates for a unit disc.

Parameters

`n` Radial index (must be non-negative).

`m` Azimuthal index ($|m| \leq n$, same parity as n).

`rho`
Normalized radial coordinate ($0 \leq \rho \leq 1$).

`phi`
Azimuthal angle in radians.

`normalisation`
Normalisation type: None (unit peak amplitude on the disc, the default), "intZ" or "Noll". "maxZ" is deprecated.

`n0`
Threshold for switching to recurrence method. Default 25.

`outside`
Value for points outside unit disc. Default 0, use `np.nan` for plotting.

`_outside_mask`
Optional precomputed boolean mask ($\rho > 1$) for speed.

`_rho_powers`
Optional precomputed dict of {power: `rho**power`} for speed.

`_azimuthal`
Optional precomputed dict of {m: $\cos$ / $\sin(|m|\phi)$} for speed.

Returns

Zernike polynomial values with same shape as `rho` and `phi`.

Raises

`ValueError`
If $|m| > n$ (invalid Zernike index).

Example

```
>>> rho, phi = create_polar_grid(np.zeros((101, 101)))
>>> Z20 = Znm(2, 0, rho, phi)  # Defocus
```

Rnm

function

```
Rnm(n: int, m: int, rho: ndarray, n0: int = 25, _rho_powers: dict = None) -> ndarray
```

Compute the radial component of the Zernike polynomial.

Parameters

`n` Radial index (must be non-negative).

`m` Azimuthal index (only $|m|$ is used).

`rho`
: Normalized radial coordinate.

`n0`
: Threshold for switching to recurrence method. Default 25.

`_rho_powers`
: Optional precomputed dict of {power: `rho**power`} for speed.

Returns

Radial polynomial values with same shape as `rho`.

Raises

`ValueError`
: If $n < 0$.

Notes

Returns zero for odd $(n - m)$. For $n > n_0$, uses Chong–Kintner recurrence.

Rnm_recurrence

function

```
Rnm_recurrence(n: int, m: int, rho: ndarray, averageNaNs: bool = True,
    quietmode: bool = False) -> ndarray
```

Compute radial Zernike polynomial using Chong–Kintner recurrence relations.

This method avoids factorial overflow for high-order polynomials.

Parameters

`n` Radial index (must be non-negative).

`m` Azimuthal index (only $|m|$ is used).

`rho`
: Normalized radial coordinate.

`averageNaNs`
: Replace NaNs from $\rho = 0$ with neighbor average. Default True.

`quietmode`
Suppress print output. Default False.

Returns

Radial polynomial values with same shape as `rho`.

Raises

`ValueError`
If $n < 0$.

Notes

Based on SimTools `FT_Rnm_recurrence.m` (C. Bond).

Anm function

```
Anm(m: int, phi: ndarray) -> ndarray
```

Compute the azimuthal component of the Zernike polynomial.

Parameters

`m` Azimuthal index. Sign determines cos ($m \geq 0$) or sin ($m < 0$).

`phi`
Azimuthal angle in radians.

Returns

$\cos(|m|\phi)$ if $m \geq 0$, $\sin(|m|\phi)$ if $m < 0$.

4.3.2 Index conversion

---

get_i function

```
get_i(n: int, m: int) -> int
```

Convert (n, m) Zernike indices to sequential index i.

The sequential index corresponds to the for-loop ordering: `i=0; for n in range(nmax+1): for m in range(-n, n+1, 2)`.

Parameters

`n` Radial index (non-negative).

`m` Azimuthal index ($|m| \leq n$).

Returns

Sequential index i (`int`).

Raises

`ValueError`
If $n < 0$ or $|m| > n$.

Notes

This is NOT the Noll index. Use `get_j_Noll()` for Noll ordering.

Example

```
>>> get_i(0, 0)  # Piston
0
>>> get_i(2, 0)  # Defocus
4
```

---

get_nm function

```
get_nm(i: int) -> tuple
```

Convert sequential index i to (n, m) Zernike indices.

Parameters

`i` Sequential index (non-negative).

Returns

Tuple (n, m): radial and azimuthal indices.

Raises

`ValueError`
If $i < 0$.

Example

```
>>> get_nm(0)
(0, 0)
>>> get_nm(4)
(2, 0)
```

---

get_j_Noll function

```
get_j_Noll(n: int, m: int) -> int
```

Convert (n, m) Zernike indices to Noll index j.

Parameters

`n` Radial index (non-negative).

`m` Azimuthal index.

Returns

Noll index j (starts at 1).

Raises

`ValueError`
If $n < 0$ or $|m| > n$.

---

get_nm_Noll function

```
get_nm_Noll(j: int) -> tuple
```

Convert Noll index j to (n, m) Zernike indices.

Noll indexing: even Z (cos) → even j; odd Z (sin) → odd j. Within each n, lower $|m|$ gets lower j.

Parameters

`j` Noll index (starts at 1, not 0).

Returns

Tuple (n, m): radial and azimuthal indices.

Raises

`ValueError`
If $j < 1$.

Notes

Based on VIR-0690A-23 Eq. 4.4.

---

list_nm function

```
list_nm(znm: ndarray) -> tuple
```

Get arrays of n and m indices for all terms in a coefficient array.

Parameters

`znm`
1D array of Zernike coefficients (i-indexed).

Returns

Tuple `(n_array, m_array)` with `n=[0,1,1,2,2,2,...]`, `m=[0,-1,1,-2,0,2,...]`.

Example

```
>>> n, m = list_nm(np.zeros(6))
>>> n
array([0, 1, 1, 2, 2, 2])
```

Noll_order_basis function

```
Noll_order_basis(znm: ndarray) -> ndarray
```

Reorder a coefficient array from i-indexing to Noll j-indexing.

Parameters

`znm`
: Coefficient array with i-indexing.

Returns

Reordered array where index `[j-1]` contains coefficient for Noll j.

---

get_Znm_name function

```
get_Znm_name(n: int, m: int) -> str
```

Get the human-readable name of a Zernike polynomial.

Parameters

`n` Radial index ($0 \leq n \leq 5$ for named terms).

`m` Azimuthal index.

Returns

Name like “Focus”, “Coma X” for $n \leq 5$, or “Z_n,m” for $n > 5$.

Raises

`ValueError`
: If $n < 0$ or $|m| > n$.

Example

```
>>> get_Znm_name(2, 0)
'Focus'
```

4.3.3 Map analysis

---

zernike_basis function

```
zernike_basis(nmax: int, A: ndarray, normalisation: str = None, x: ndarray = None,
    y: ndarray = None, disk_radius: float = None) -> ndarray
```

Generate a 3D array (cuboid) of Zernike polynomials up to order `nmax`.

Parameters

`nmax`
: Maximum radial order to include.

`A` 2D reference array defining the grid shape.

`normalisation`
Normalization type passed to `Znm()`. Default None.

`x` 1D array of x-coordinates (physical units, e.g., meters). If provided with `y` and `disk_radius`, uses physical coordinates.

`y` 1D array of y-coordinates (physical units, e.g., meters).

`disk_radius`
Radius defining the unit disc (same units as x, y). If larger than data extent, basis extends beyond data. If smaller, data outside `disk_radius` is masked.

Returns

3D array of shape `(imax+1, A.shape[0], A.shape[1])`.

Raises

`ValueError`
If only some of `x`, `y`, `disk_radius` are provided.

Example

```
>>> A = np.zeros((256, 256))
>>> basis = zernike_basis(nmax=5, A=A)
>>> basis.shape
(21, 256, 256)

>>> # Fit over a larger physical disc than the data extent:
>>> basis = zernike_basis(nmax=10, A=data,
...                       x=x_coords, y=y_coords, disk_radius=0.175)
```

---

get_cnm

function

```
get_cnm(mapdata: ndarray, basis: ndarray, scaling: str = 'basis') -> ndarray
```

Decompose map data into Zernike coefficients (fast, assumes orthogonality).

Computes each coefficient independently using projection:

$$c_i = \frac{\sum Z_i \cdot \text{data}}{\sum Z_i^2}.$$

This is fast ($O(n)$ in number of terms) but assumes basis functions are orthogonal over the data region.

Use this function when:

- fitting over the full unit disc with no missing data;
- speed is critical (many terms or repeated calls).

Use `get_cnm_robust()` instead when:

- data does not cover the full disk, e.g. has NaN values in the given radius;
- fitting over a subset of the unit disc;
- accuracy matters more than speed.

Notes

Zernike polynomials are only orthogonal over the full unit disc. When fitting over masked regions, basis functions become correlated and this function gives incorrect coefficients. In such cases, `get_cnm_robust()` solves the full linear system correctly.

Parameters

`mapdata`
: 2D map data to decompose. May contain NaN values, but see note.

`basis`
: 3D Zernike basis from `zernike_basis()`.

`scaling`
: Coefficient scaling method: 'basis' (default) or 'pi'.

Returns

1D array of coefficients (i-indexed).

Raises

`ValueError`
: If `scaling` is not 'basis' or 'pi'.

Example

```
>>> basis = zernike_basis(nmax=5, A=surface)
>>> cnm = get_cnm(surface, basis)  # Full disc, no masking
```

---

get_cnm_robust function

```
get_cnm_robust(mapdata: ndarray, basis: ndarray, block: int = 65536) -> ndarray
```

Decompose map data into Zernike coefficients using least-squares.

Unlike `get_cnm()`, this function correctly handles cases where the basis functions are not orthogonal over the data region (e.g., when parts of the map are masked with NaN). It solves the full linear system rather than computing each coefficient independently.

The standard projection formula $c_i = \int Z_i \cdot \text{data} \,/\, \int Z_i^2$ assumes orthogonality: $\int Z_i Z_j = 0$ for $i \neq j$. When this doesn't hold (e.g., fitting over a subset of the unit disc), this function solves

$$G\,c = b,$$

where $G_{ij} = \sum Z_i Z_j$ is the Gram matrix and $b_i = \sum Z_i \cdot \text{data}$.

Parameters

`mapdata`
: 2D map data to decompose. May contain NaN values.

`basis`
: 3D Zernike basis from `zernike_basis()`.

`block`
: Number of pixels per block in the accumulation of the normal equations. The transient copy of the basis is `n_terms * block` values; the default is 34 MB for 66 terms.

Returns

1D array of coefficients (i-indexed).

Notes

The Gram matrix and the projection vector are accumulated with matrix products over blocks of valid pixels, so the cost is a fraction of a second for a 927×927 map at order 10. Use `get_cnm()` when fitting over the full disc, where the terms are orthogonal.

See also: Mahajan, V.N. "Zernike polynomials and wavefront fitting" in Optical Shop Testing, 3rd ed. (Malacara, ed.), Wiley, 2007, Ch. 13, Sec. 13.5 (discusses loss of Zernike orthogonality over discrete or non-circular data and coefficient recovery). Strang, G. "Introduction to Linear Algebra", MIT Press, Ch. 4 (derives the normal equations $A^T Ax = A^T b$ (Gram matrix solve) as the solution to least-squares minimisation of $\|Ax - b\|^2$).

Example

```
>>> # When outer region is masked, use robust version:
>>> basis = zernike_basis(nmax=5, A=data)
>>> cnm = get_cnm_robust(data_with_nans, basis)
```

`fit_zernike` function

```
fit_zernike(x: ndarray, y: ndarray, Z: ndarray, nmax: int, disk_radius: float | None = None,
    normalisation: str = None, method: str = 'projection')
    -> tuple[ndarray, ndarray, ndarray]
```

Fit Zernike polynomials to a map in one call.

Builds the basis up to radial order `nmax` on the physical grid, decomposes the map and returns the coefficients together with their radial and azimuthal indices, in the i order of `get_i()`. This bundles `zernike_basis()`, `get_cnm()` and `list_nm()` for the common case of characterising a map.

Parameters

`x` 1D array of x-coordinates [m].

`y` 1D array of y-coordinates [m].

`Z` 2D map with shape `(len(y), len(x))`, NaN outside the data.

`nmax`
: Maximum radial order.

`disk_radius`
: Radius of the Zernike unit disc [m]. Default None uses the radius of the data: the largest radius at which `Z` is finite, at most the largest centred circle inside the grid.

`normalisation`
: Normalisation of the polynomials, see `Znm()`. Default None.

`method`
: "projection" (default) computes each coefficient by projection with `get_cnm()`, which assumes that the data covers the disc. "lstsq" solves the full linear system with `get_cnm_robust()`, correct for partial coverage.

Returns

Tuple `(cnm, n, m)`: the coefficients in the unit of `Z` and the integer index arrays of the same

length.

Raises

`ValueError`
If `method` is unknown or `Z` has no finite values.

Example

```
>>> cnm, n, m = fit_zernike(x, y, Z, nmax=14, disk_radius=0.175)
>>> defocus = cnm[(n == 2) & (m == 0)][0]
```

---

make_zernike_map function

```
make_zernike_map(basis: ndarray, cnm: ndarray) -> ndarray
```

Generate a map from Zernike basis and coefficients.

Parameters

`basis`
3D Zernike basis from `zernike_basis()`.

`cnm`
1D array of coefficients (i-indexed).

Returns

2D map constructed as sum of `cnm[i] * basis[i]`.

Notes

Ensure coefficients were computed with the same normalization as the basis. The sum is evaluated term by term, without a copy of the basis.

---

rms_from_cnm function

```
rms_from_cnm(cnm: ndarray) -> float
```

Calculate RMS surface error from Zernike coefficients.

Computes RMS excluding the piston term (which represents constant offset). Based on VIR-0690A-23 Eq. 4.9:

$$\mathrm{rms}^2 = \sum_{j \geq 2} c_j^2.$$

Parameters

`cnm`
1D array of Zernike coefficients (i-indexed).

Returns

RMS value `sqrt(sum(cnm[1:]**2))`.

Example

```
>>> cnm = np.array([1.0, 0.1, 0.1, 0.05])  # piston + aberrations
>>> rms = rms_from_cnm(cnm)  # Excludes piston
```

---

remove_ptc_zernike

function

```
remove_ptc_zernike(x: ndarray, y: ndarray, Z: ndarray, piston: bool = True,
    tilt: bool = True, roc: bool = False, disk_radius: float = None,
    debug: bool = False) -> tuple
```

Remove piston, tilt, and/or curvature using Zernike polynomial fitting.

Fits low-order Zernike polynomials to the map and subtracts the specified terms. Unlike `remove_ptc_finesse` (Gaussian-weighted overlap), this function is beam-size independent — it performs a geometric fit over the data aperture defined by `disk_radius`.

When the data contains NaN regions (e.g. partial-aperture metrology), `get_cnm_robust` (least-squares) is used automatically instead of the fast projection `get_cnm`, so that loss of Zernike orthogonality over the masked region is handled correctly.

Parameters

`x` 1D array of x-coordinates (physical units, e.g. metres).

`y` 1D array of y-coordinates (physical units, e.g. metres).

`Z` 2D map data with shape `(len(y), len(x))`. May contain NaN.

`piston`
: Remove piston (Z_0^0). Default True.

`tilt`
: Remove tilt (Z_1^{-1}, Z_1^1). Default True.

`roc`
: Remove defocus/curvature (Z_2^0). Default False.

`disk_radius`
: Radius defining the Zernike unit disc (same units as x/y). If None, the disc is inferred from the array shape (original behaviour).

`debug`
: If True, print removed values and show diagnostic plots. Default False.

Returns

Tuple `(x, y, corrected_map)` with specified aberrations removed. NaN values are preserved at their original positions.

Notes

Astigmatism terms (Z_2^{-2}, Z_2^2) are always preserved — they represent real mirror features, not alignment degrees of freedom.

Example

```
>>> x, y, A_flat = remove_ptc_zernike(x, y, surface,
...     piston=True, tilt=True, roc=True, disk_radius=0.175)
```

zero_cnm_terms function

```
zero_cnm_terms(cnm: ndarray, nm: list = None) -> ndarray
```

Return a copy of coefficients with specified terms set to zero.

Parameters

`cnm`
: 1D array of coefficients (i-indexed).

`nm`
: Terms to zero. Can contain int (zero all m for that n) or tuple (n, m) for specific term.

Returns

Copy of `cnm` with specified terms zeroed.

Example

```
>>> cnm = np.ones(10)
>>> cnm_zeroed = zero_cnm_terms(cnm, nm=[0, 1, (2, 0)])
```

4.3.4 Random generation

get_Bn function

```
get_Bn(cnm: ndarray, method: str = 'quad', skipnm: list = None) -> ndarray
```

Compute layer coefficients B_n from Zernike coefficients.

Aggregates all m terms for each radial order n into a single coefficient.

Parameters

`cnm`
: 1D array of Zernike coefficients.

`method`
: Aggregation method — 'quad' (sqrt of sum of squares) or 'abs' (sum of absolute values). Default 'quad'.

`skipnm`
: Terms to exclude before computing B_n. Common: `[0, 1, (2,0)]` to skip piston, tilt, defocus.

Returns

1D array of layer coefficients, one per radial order.

Example

```
>>> Bn = get_Bn(cnm, method='quad', skipnm=[0, 1, (2, 0)])
```

random_cnm_flat function

```
random_cnm_flat(nmax: int, skipnm: list = None, sf: float = 1) -> ndarray
```

Generate random Zernike coefficients with uniform distribution.

Parameters

`nmax`
: Maximum radial order.

`skipnm`
: Terms to set to zero.

`sf`
: Scale factor. Default 1 gives $U(-0.5, 0.5)$.

Returns

Random coefficients (i-indexed).

random_cnm_Bn function

```
random_cnm_Bn(Bn: ndarray, skipnm_norm: list = None) -> ndarray
```

Generate random coefficients weighted by layer coefficients (quadrature).

Creates random coefficients that statistically reproduce the layer coefficient structure of a reference map. Based on Tao et al. PRD 2020.

Parameters

`Bn`
: Layer coefficients from `get_Bn(method='quad')`.

`skipnm_norm`
: Terms to exclude from normalization.

Returns

Random coefficients (i-indexed).

random_cnm_simtools function

```
random_cnm_simtools(Bn: ndarray, skipnm_norm: list = None) -> ndarray
```

Generate random coefficients weighted by layer coefficients (absolute sum).

Similar to `random_cnm_Bn` but uses absolute sum for layer coefficients, matching the SimTools implementation (C. Bond thesis 2014).

Parameters

`Bn`
: Layer coefficients from `get_Bn(method='abs')`.

`skipnm_norm`
Terms to exclude from normalization.

Returns

Random coefficients (i-indexed).

4.3.5 Visualization

---

plot_Znm function

```
plot_Znm(n: int = None, m: int = None, i: int = None, npx: int = 101,
    colormap: str = 'jet', ax=None, fsize: int = 16, padtxt: float = 0.07,
    debug: bool = False)
```

Plot a single Zernike polynomial.

Parameters

`n` Radial index (required if `i` not provided).

`m` Azimuthal index (required if `i` not provided).

`i` Sequential index (alternative to `n`, `m`).

`npx`
Number of pixels per side. Default 101.

`colormap`
Matplotlib colormap name. Default 'jet'.

`ax`
Matplotlib axes to plot on.

`fsize`
Font size for label. Default 16.

`padtxt`
Text padding from corner. Default 0.07.

`debug`
Print debug information. Default False.

---

plot_zernike_pyramid function

```
plot_zernike_pyramid(nmax: int = 3, figsize: float = 2.5, fontsize: int = 12,
    pad: float = 0.05, pad_text: float = 0.07, npixels: int = 301, colormap: str = 'jet')
```

Plot a pyramid of Zernike polynomials up to order `nmax`.

Parameters

`nmax`
Maximum radial order to display.

`figsize`
Overall figure size.

`fontsize`
Font size for labels.

`pad`
Subplot padding.

`pad_text`
Text label padding.

`npixels`
Resolution of each subplot.

`colormap`
Matplotlib colormap name.

---

plot_basis

function

```
plot_basis(basis: ndarray, figsize: float = 2.5, pad: float = 0.05, colormap: str = 'jet',
    padtxt: float = 0.12, fsize: int = 8)
```

Plot a pyramidal figure showing all terms in a Zernike basis.

Parameters

`basis`
3D basis array from `zernike_basis()`.

`figsize`
Overall figure size.

`pad`
Subplot padding.

`colormap`
Matplotlib colormap name.

`padtxt`
Text label padding.

`fsize`
Font size for index labels.

---

make_pyramid_figure

function

```
make_pyramid_figure(N: int = 3, figsize: float = 2.5, pad: float = 0.05)
```

Create a figure with pyramidal subplot arrangement for Zernike display.

Parameters

`N` Number of rows in the pyramid.

`figsize`
Overall figure size (square).

`pad`
Padding between subplots as fraction of subplot size.

Returns

Matplotlib figure with pyramid arrangement of axes.

---

plot_cnm_histogram

function

```
plot_cnm_histogram(cnm: ndarray, absolute: bool = False, **kwargs)
```

Plot a histogram of Zernike coefficients.

Parameters

`cnm`
: 1D array of coefficients.

`absolute`
: If True, plot absolute values.

`**kwargs`
: Additional arguments passed to `utils.plot_histogram()`.

Returns

Matplotlib axes object.

---

plot_Bn_histogram

function

```
plot_Bn_histogram(Bn: ndarray, **kwargs)
```

Plot a histogram of layer coefficients.

Parameters

`Bn`
: 1D array of layer coefficients.

`**kwargs`
: Additional arguments passed to `utils.plot_histogram()`.

Returns

Matplotlib axes object.

4.4 Hermite-Gauss modes: `mirror_maps.hermite_gauss`

Mirror map analysis and reconstruction using Hermite-Gauss Mode
Felice Feldmann, Andreas Freise, Anna Green, March 2026

Tools for decomposing mirror surface maps into Hermite-Gauss (HG) mode coefficients and reconstructing maps from those coefficients. Designed to complement the Zernike analysis in `zernike.py`.

Key features

- HG basis generation.
- Coefficient computation via projection (with circular aperture masking).
- Map reconstruction from HG coefficients.
- Consistent API with the Zernike module.

Main functions

Function	Purpose
`hg_basis`	Generate 3D basis cuboid of HG mode fields
`hg_mode_indices`	List (n, m) pairs for a given `nmax`
`get_hg_coeffs`	Decompose map into HG coefficients
`get_hg_coeffs_wls`	Weighted least-squares HG coefficients
`make_hg_map`	Reconstruct map from coefficients
`coupling_coefficients`	Specific mirror optical coupling $c_{00\to nm}$
`scattering_loss`	Single-bounce loss $1 - \lvert k_{00}\rvert^2$ of the fundamental mode

Technical notes

- Mode ordering follows `for n in range(nmax+1): for m in range(nmax-n+1)`, so the total mode order $n+m$ never exceeds `nmax`. The number of modes is $(n_{\max}+1)(n_{\max}+2)/2$.
- Modes are evaluated at $z = 0$ (beam waist) and are therefore real-valued.
- A circular aperture of radius $R_{\max}$ is applied to both the basis and the map data during projection, matching the physical mirror extent.
- The beam waist w_0 controls the trade-off between centre accuracy and edge coverage. A useful heuristic is $w_0 \approx 1.5\, R_{\max}/\sqrt{2n_{\max}+1}$ (based on Carter 1979).

Requires: `numpy`, `finesse`.

4.4.1 Basis and decomposition

---

hg_basis function

```
hg_basis(nmax: int, x: ndarray, y: ndarray, w0: float, Rmax: float | None = None) -> ndarray
```

Generate a 3D basis cuboid of real-valued HG mode fields.

Each slice `basis[i]` contains the real part of HG mode (n, m) evaluated on the (x, y) grid at $z = 0$ (beam waist). An optional circular aperture zeros out the field beyond radius `Rmax`.

Parameters

`nmax`
: Maximum total mode order ($n + m \le n_{\max}$).

`x` 1D array of x-coordinates [m].

`y` 1D array of y-coordinates [m].

`w0`
Beam waist radius [m].

`Rmax`
Aperture radius [m]. Points with $R > R_{\max}$ are set to zero. Default None (no masking).

Returns

3D array of shape `(n_modes, len(y), len(x))`, where `n_modes = (nmax+1)*(nmax+2)/2`.

Example

```
>>> x = np.linspace(-0.15, 0.15, 512)
>>> basis = hg_basis(nmax=10, x=x, y=x, w0=0.04, Rmax=0.15)
>>> basis.shape
(66, 512, 512)
```

---

hg_mode_indices

function

```
hg_mode_indices(nmax: int) -> list[tuple[int, int]]
```

List all (n, m) index pairs for HG modes up to total order `nmax`.

The ordering is: for each n from 0 to `nmax`, m runs from 0 to `nmax`$-n$. This means $n+m \leq n_{\max}$ for every mode.

Parameters

`nmax`
Maximum total mode order ($n + m \leq n_{\max}$).

Returns

List of (n, m) tuples.

Example

```
>>> hg_mode_indices(2)
[(0, 0), (0, 1), (0, 2), (1, 0), (1, 1), (2, 0)]
```

---

get_hg_coeffs

function

```
get_hg_coeffs(mapdata: ndarray, basis: ndarray) -> ndarray
```

Decompose map data into HG coefficients by projection.

Computes each coefficient as

$$c_i = \frac{\sum \text{basis}_i \cdot \text{mapdata}}{\sum \text{basis}_i^2}.$$

This normalisation (identical to `zernike.get_cnm` with `scaling='basis'`) makes the result independent of the grid spacing and gives good reconstructions even though the HG modes are not strictly orthogonal over a finite circular aperture.

NaN values in `mapdata` are treated as zero during projection.

Parameters

`mapdata`
2D map data with shape `(len(y), len(x))`. May contain NaN.

`basis`
3D basis from `hg_basis()`, shape `(n_modes, ny, nx)`.

Returns

1D array of coefficients, length `n_modes`.

Notes

The basis must have been generated on the same (x, y) grid as `mapdata`, and with the desired aperture already applied.

Example

```
>>> coeffs = get_hg_coeffs(surface_nm, basis)
```

`get_hg_coeffs_wls` function

```
get_hg_coeffs_wls(mapdata: ndarray, basis: ndarray, weight: ndarray) -> ndarray
```

Weighted least-squares HG coefficients.

Minimises $\sum \text{weight} \cdot (\text{mapdata} - \text{reconstruction})^2$ over the coefficient vector. This allows the fit to prioritise the beam centre (using e.g. a Gaussian beam-intensity weight) rather than treating all pixels equally.

Parameters

`mapdata`
2D map data, shape `(ny, nx)`. NaN treated as zero.

`basis`
3D basis from `hg_basis()`, shape `(n_modes, ny, nx)`.

`weight`
2D weight array, shape `(ny, nx)`. For beam-intensity weighting use $\exp(-2R^2/w_{\text{gauss}}^2)$.

Returns

1D array of coefficients, length `n_modes`.

Example

```
>>> weight = np.exp(-2 * R**2 / 0.080**2)
>>> coeffs = get_hg_coeffs_wls(surface_nm, basis, weight)
```

make_hg_map function

```
make_hg_map(basis: ndarray, coeffs: ndarray) -> ndarray
```

Reconstruct a 2D map from HG basis and coefficients.

Parameters

`basis`
: 3D basis from `hg_basis()`, shape `(n_modes, ny, nx)`.

`coeffs`
: 1D array of coefficients from `get_hg_coeffs`.

Returns

2D reconstructed map, shape `(ny, nx)`.

Example

```
>>> reconstructed = make_hg_map(basis, coeffs)
```

4.4.2 Coupling and loss

coupling_coefficients function

```
coupling_coefficients(x: ndarray, y: ndarray, Z_nm: ndarray, beam_spot: float,
    wavelength: float = 1.064e-06, nmax_out: int = 8)
    -> tuple[list[tuple[int, int]], ndarray]
```

Mirror reflection coupling coefficients $c_{00\to nm}$.

Computes how a reflected fundamental Gaussian beam couples into higher-order HG modes due to the mirror surface figure $Z(x, y)$.

Uses separability of HG modes — two matrix multiplications replace a Python loop over 2D overlaps, giving a significant speed-up.

Parameters

`x, y`
: 1D coordinate arrays [m].

`Z_nm`
: 2D surface height map [nm].

`beam_spot`
: Beam spot size ($1/e^2$ intensity radius) [m].

`wavelength`
: Laser wavelength [m]. Default 1064 nm.

`nmax_out`
: Maximum mode order for output coupling.

Returns

`indices`
List of (n, m) tuples.

`coeffs`
Complex coupling coefficients $c_{00\to nm}$.

Example

```
>>> indices, c = coupling_coefficients(x, y, mapdata, 0.056)
>>> power_loss_ppm = (1 - abs(c[0])**2) * 1e6
```

`scattering_loss` function

```
scattering_loss(x: ndarray, y: ndarray, Z_nm: ndarray, beam_radius: float,
    wavelength: float = 1.064e-06, mirror_radius: float | None = None) -> float
```

Single-bounce scattering loss of the fundamental mode, $1 - |k_{00}|^2$.

The reflected fundamental Gaussian beam is projected onto the incident one, giving $k_{00} = c_{00\to 00}$ of `coupling_coefficients()`, and the power missing from that overlap is returned as a fraction; multiply by 10^6 for ppm. The map is used as given. (If needed remove piston, tilt and curvature with `utils.apply_ptc_finesse()` first.)

Pixels with a non-finite height (NaN or inf) are treated as zero. When `mirror_radius` is given, the surface outside that radius is also treated as zero height. The mirror radius in this function does not apply a clipping aperture.

Parameters

`x, y`
1D coordinate arrays [m], uniformly spaced.

`Z_nm`
2D surface height map [nm] with shape `(len(y), len(x))`.

`beam_radius`
Beam radius ($1/e^2$ intensity) on the mirror [m].

`wavelength`
Laser wavelength [m]. Default 1064 nm.

`mirror_radius`
Radius of the mirror aperture [m]. Default None applies no mask.

Returns

The loss $1 - |k_{00}|^2$, dimensionless (`float`).

Example

```
>>> Zw = apply_ptc_finesse(x, y, Z, 0.12, n_iter=4)
>>> loss_ppm = 1e6 * scattering_loss(x, y, Zw, 0.12, mirror_radius=0.31)
```

4.5 Power spectral densities: `mirror_maps.psd`

Power spectral density analysis and virtual map generation. FFT-based toolkit for analyzing optical surface maps and generating synthetic surfaces with prescribed statistical properties.

Designed for characterizing mirror surface roughness, optical aberrations, and creating virtual test maps for interferometric analysis.

Key features

- 1D and 2D power spectral density (PSD) computation with windowing.
- Radial averaging for converting 2D PSDs to 1D profiles.
- Inverse PSD synthesis for generating random maps with specified statistics.
- Circular and Tukey windowing for spatial domain tapering.
- Low-pass frequency filtering for surface smoothing.
- Visualization tools for FFT and PSD data.

Main functions

Function	Purpose
PSD analysis	
`psd`	1D power spectral density with Hanning window
`psd2D`	2D power spectral density with circular window
`map_to_psd1D`	Combined 2D PSD computation and radial averaging
`radial_average_psd`	Angular averaging of 2D PSD to 1D radial profile
RMS calculations	
`psd1D_rms`	Estimate RMS from 1D radially-averaged PSD
`psd2D_rms`	Calculate RMS from 2D PSD using Parseval's theorem
Map synthesis and coordinate transformation	
`psd1D_to_map`	Generate random 2D surface from 1D radial PSD (rfft2 only)
`psd1D_to_2D`	Extrapolate 1D radial PSD to 2D with circular symmetry
Windowing and filtering	
`tukey_window`	Circular Tukey window generation
`apply_circular_window`	Apply Hanning window to 2D map
`apply_lowpass_filter_2D`	Circular low-pass filter in frequency domain
Visualization	
`plot_psd2D`	Display 2D FFT/PSD with log colormap
`plot_psd1D`	Display 1D PSD as linear spectral density
`psd2D_cross_section`	Extract 1D cross-section through 2D PSD

Technical notes

- All PSD computations use proper normalization for Parseval's theorem.
- Window functions include normalization factors (S_2) for accurate power.
- Single-sided spectra (rfft) include selective factoring for power conservation.
- `PSD_to_map` always uses `irfft2` (single-sided format) for efficiency.
- Generated maps maintain specified RMS values within circular apertures.

- Coordinate systems assume square grids with equal spacing ($dx = dy$).

Normalisation conventions

- `psd2D(single=True)` returns the single-sided 2D PSD: $f_x \geq 0$, with the $f_x > 0$ columns doubled, so that its sum times $df_x\, df_y$ recovers the variance of the windowed, mean-removed map.
- `radial_average_psd()` undoes that doubling before averaging: the returned 1D PSD is the azimuthal mean of the double-sided 2D PSD as a function of radial frequency f_r. The map variance is recovered as

  $$\sigma^2 = 2\pi \int \mathrm{psd1d}(f_r)\, f_r\, \mathrm{d}f_r$$

  over the full frequency plane (`psd1D_rms`; exact with the `ring_power` output), not as a plain integral over f_r.
- `psd1D_to_map()` inverts the same convention, so the analysis and synthesis functions round-trip consistently (see `tests/mirror_maps/test_psd_regression.py`).
- With a window, the recovered variance is the window-weighted mean square of the mean-removed map. For maps whose variance is concentrated at the aperture edge, where the window goes to zero, this is smaller than the plain aperture variance.

Dependencies

Requires: numpy, scipy, matplotlib, finesse, h5py

Authors

Andreas Freise, Anna Green January 2026

Examples

```
>>> # Analyze a mirror surface map
>>> fx, fy, psd2d = psd2D(x, y, surface_data, window=True, single=True)
>>> fr, psd1d, std = radial_average_psd(fx, fy, psd2d, single=True)
>>>
>>> # Generate a synthetic surface with same statistics
>>> x_new, y_new, virtual_map = psd1D_to_map(fr, psd1d, width=0.1, points=512)
```

4.5.1 PSD analysis

---

psd function

```
psd(x: ndarray, y0: ndarray, window: bool = True, single: bool = True,
    debug: bool = False) -> tuple[ndarray, ndarray]
```

Compute 1D power spectral density.

Parameters

`x` 1D coordinate axis.

`y0`
1D data array.

`window`
Apply Hanning window. Default True.

`single`
Return single-sided spectrum. Default True.

`debug`
Print RMS cross-checks. Default False.

Returns

Tuple `(f, PSD)`: frequency axis [1/units of x] and power spectral density.

Notes

Removes mean before computation. Single-sided spectrum uses rfft with factor of 2 normalization.

---

psd2D

function

```
psd2D(x: ndarray, y: ndarray, A: ndarray, window: bool = True,
    window_radius: float | None = None, single: bool = True, debug: bool = False)
    -> tuple[ndarray, ndarray, ndarray]
```

Compute 2D power spectral density of map data.

Parameters

`x, y`
1D coordinate axes defining the physical grid dimensions.

`A` 2D map data.

`window`
Apply circular Hanning window before FFT. Default True.

`window_radius`
Radius for windowing. Default None uses full extent.

`single`
Return single-sided spectrum (rfft2). Default True.

`debug`
Print RMS cross-checks. Default False.

Returns

Tuple `(fx, fy, PSD)`: frequency axes [1/units of x,y] and 2D PSD.

Notes

Single-sided mode uses rfft2, returning positive frequencies in x and full spectrum in y (fftshift applied). Applies selective factoring (`PSD[:, 1:] *= 2`) for single-sided spectra to ensure energy conservation.

The map mean is removed before (and, with a window, again after) windowing. With `window=True` the sum of the PSD times $df_x\, df_y$ equals the window-weighted mean square of

the mean-removed map (normalisation by S_2 = sum of the squared window). For maps whose variance is concentrated near the aperture edge, where the window goes to zero, this is smaller than the plain aperture variance.

---

radial_average_psd

function

```
radial_average_psd(fx: ndarray, fy: ndarray, A: ndarray, single: bool = True,
    debug: bool = False) -> tuple[ndarray, ndarray, ndarray, ndarray]
```

Compute 1D radial average of 2D PSD by averaging over angle.

Parameters

`fx, fy`
: 2D frequency axes (must have equal spacing and extent).

`A`
: 2D PSD data to average.

`single`
: If True, corrects for selective factoring from `psd2D(single=True)` by dividing $k_x > 0$ columns by 2 before averaging. Default True.

`debug`
: Print diagnostic information. Default False.

Returns

Tuple `(fr, psd1d, psd1d_std, ring_power)`: radial frequency axis, radially averaged 1D PSD, standard deviation in each radial bin, and total power per ring (sum of PSD $\cdot\, df_x\, df_y$ for each ring). The `ring_power` array can be passed to `psd1D_rms` for exact discrete RMS computation.

Notes

Requires `fx` and `fy` to have identical spacing ($df_x = df_y$) and extent. The `single` parameter should match the value used in `psd2D`.

With `single=True` the factor of 2 applied to the $f_x > 0$ columns in `psd2D` is undone before averaging, so the returned `psd1d` is the azimuthal mean of the double-sided 2D PSD at radial frequency f_r. The map variance is recovered as $\sigma^2 = 2\pi \int \mathrm{psd1d}(f_r)\, f_r\, \mathrm{d}f_r$ (see `psd1D_rms`), not as a plain integral over f_r.

---

map_to_psd1D

function

```
map_to_psd1D(x: ndarray, y: ndarray, A: ndarray, is_single: bool = True,
    debug: bool = False) -> tuple[ndarray, ndarray]
```

Convenience function combining `psd2D` and radial average.

Parameters

`x, y`
: 1D coordinate axes defining the grid.

`A`
: 2D map data.

`is_single`
: Use single-sided spectrum. Default True.

`debug`
: Print diagnostic information. Default False.

Returns

Tuple `(fr, pr)`: radial frequency axis and radially averaged 1D PSD.

Notes

The returned `pr` follows the `radial_average_psd` convention: it is the azimuthal mean of the double-sided 2D PSD, and the map variance is recovered as $\sigma^2 = 2\pi \int \mathrm{pr}(f_r)\, f_r \, \mathrm{d}f_r$ (see `psd1D_rms`).

`psd2D_cross_section` function

```
psd2D_cross_section(fx: ndarray, fy: ndarray, psd2d: ndarray, axis: str = 'x')
    -> tuple[ndarray, ndarray]
```

Extract a 1D cross-section through a 2D PSD.

Takes a slice at $f_y = 0$ (`axis='x'`) or $f_x = 0$ (`axis='y'`), returning only positive frequencies. Useful for comparing radially-averaged 1D PSD with direct cross-sections.

Parameters

`fx`
: 1D frequency axis in x-direction from `psd2D`.

`fy`
: 1D frequency axis in y-direction from `psd2D`.

`psd2d`
: 2D PSD array from `psd2D`.

`axis`
: Direction of cross-section: 'x' (at $f_y = 0$) or 'y' (at $f_x = 0$). Default 'x'.

Returns

Tuple `(f_cross, psd_cross)`: frequency and PSD values along cross-section.

Raises

`ValueError`
: If `axis` is not 'x' or 'y'.

Example

```
>>> f_cross, psd_cross = psd2D_cross_section(fx, fy, psd2d, axis='x')
>>> plt.loglog(f_cross, np.sqrt(psd_cross), label='Cross-section')
```

4.5.2 RMS calculations

psd1D_rms

function

```
psd1D_rms(fr: ndarray, psd1d: ndarray, ring_power: ndarray | None = None) -> float
```

Estimate RMS from a 1D radially-averaged PSD.

Parameters

`fr`
: 1D array of radial spatial frequencies (uniform spacing assumed).

`psd1d`
: 1D array of PSD values at those frequencies, as returned by `radial_average_psd(single=True)`.

`ring_power`
: Optional array of per-ring power from `radial_average_psd`. When provided, uses exact discrete pixel counts instead of the continuous ring-area approximation $2\pi f_r\, df_r$.

Returns

Estimated RMS value (`float`).

Notes

Without `ring_power`, uses the continuous polar-coordinate approximation

$$\mathrm{RMS}^2 = 2\pi \int \mathrm{PSD}_{1\mathrm{D}}(f_r)\, f_r\, \mathrm{d}f_r \approx 2\pi\, df_r \sum f_r \cdot \mathrm{psd1d}.$$

This underestimates the true RMS because the continuous ring area $2\pi f_r\, df_r$ differs from the actual discrete pixel count, especially at the lowest frequencies where steep PSDs have most power.

With `ring_power` (from `radial_average_psd`), the RMS is exact:

$$\mathrm{RMS}^2 = \sum_i \mathrm{ring_power}_i.$$

The `psd1d` values must come from `radial_average_psd(single=True)`, which undoes the factor-of-2 applied in `psd2D` for the single-sided spectrum. Reference: VIR-0690A-23.

Example

```
>>> fr, psd1d, _, ring_power = radial_average_psd(fx, fy, psd2d, single=True)
>>> rms = psd1D_rms(fr, psd1d, ring_power=ring_power)
```

psd2D_rms

function

```
psd2D_rms(psd2d: ndarray, dfx: float, dfy: float) -> float
```

Calculate RMS from a 2D PSD using Parseval's theorem.

Parameters

`psd2d`
: 2D power spectral density array.

`dfx`
: Frequency resolution in x-direction [1/m].

`dfy`
Frequency resolution in y-direction [1/m].

Returns

RMS value computed from the 2D PSD (`float`).

Notes

By Parseval's theorem: RMS $= \sqrt{\sum \mathrm{PSD} \cdot df_x\, df_y}$. Works with both single-sided (rfft2) and double-sided (fft2) PSDs.

When `psd2D()` is called with `window=True`, the circular Hanning window reduces effective power, so `psd2D_rms` returns the RMS of the windowed data rather than the original map. This is typically 15–20% lower than the spatial RMS computed directly from the map.

Example

```
>>> fx, fy, psd2d = psd2D(x, y, surface_data, single=True)
>>> rms = psd2D_rms(psd2d, fx[1]-fx[0], fy[1]-fy[0])
```

4.5.3 Map synthesis

`psd1D_to_map` function

```
psd1D_to_map(f: ndarray, psd: ndarray, width: float, points: int, debug: bool = False)
    -> tuple[ndarray, ndarray, ndarray]
```

Generate random 2D map from 1D radial PSD using inverse FFT.

Parameters

`f` 1D radial frequency axis.

`psd`
1D radial PSD values.

`width`
Physical width (and height) of output square map.

`points`
Number of grid points along each axis.

`debug`
Print diagnostic information. Default False.

Returns

Tuple `(x, y, B)`: coordinate axes and 2D random map with specified PSD.

Notes

Always uses `irfft2` (single-sided inverse FFT). Applies random phase, then inverse FFT. For round-trip consistency use: `psd2D(single=True)` → `radial_average_psd(single=True)` → `psd1D_to_map`.

psd1D_to_2D function

```
psd1D_to_2D(x: ndarray, y: ndarray, fr: ndarray, psd1d: ndarray, single: bool = True,
    psd1d_std: ndarray | None = None, debug: bool = False)
    -> tuple[ndarray, ndarray, ndarray]
```

Extrapolate 1D radial PSD to 2D for use with inverse FFT.

Parameters

`x, y`
: 1D coordinate axes (must define square grid with equal spacing).

`fr`
: 1D radial frequency axis.

`psd1d`
: 1D radial PSD values.

`single`
: If True (default), output single-sided format (rfft2 compatible, positive k_x only). If False, output double-sided format (fft2, full k_x range). The standard pipeline uses `single=True` throughout.

`psd1d_std`
: Standard deviation for adding random variations. Default None.

`debug`
: Print diagnostic information. Default False.

Returns

Tuple `(fx, fy, psd2d)`: frequency axes and 2D PSD.

Notes

Interpolates 1D radial PSD to create 2D PSD with circular symmetry. Sets PSD to zero outside maximum radius of input data.

4.5.4 Windowing and filtering

tukey_window function

```
tukey_window(x: ndarray, y: ndarray, alpha: float = 0.25) -> tuple[ndarray, float]
```

Create a circular Tukey window for 2D data.

Parameters

`x, y`
: 1D coordinate axes defining the grid.

`alpha`
: Tukey window taper fraction. Default 0.25.

Returns

Tuple `(w, window_radius)`: 2D window array and radius in same units as x, y.

apply_circular_window function

```
apply_circular_window(x: ndarray, y: ndarray, A: ndarray,
    window_radius: float | None = None, window_x: ndarray | None = None,
    window_data: ndarray | None = None, debug: bool = False) -> tuple[ndarray, ndarray]
```

Apply circular window to 2D map data.

Creates a 2D circular window by radially interpolating a 1D window function. Either uses a default Hanning window or a user-provided custom window.

Parameters

`x, y`
: 1D coordinate axes defining the grid.

`A`
: 2D map data to window.

`window_radius`
: Radius for default Hanning window. Default None uses `max(xlength, ylength)/2`. Ignored when custom window provided.

`window_x`
: 1D coordinate axis for custom window. Must be provided with `window_data`.

`window_data`
: 1D window function values for `window_x`. Must be provided with `window_x`.

`debug`
: Enable debug plots. Default False.

Returns

Tuple `(windowed_A, w)`: windowed 2D map and the 2D window function applied.

Notes

The 2D circular window is created by interpolating the 1D window onto $R = \sqrt{X^2 + Y^2}$. Both `window_x` and `window_data` must be provided together for custom windows.

Example

```
>>> A_windowed, w = apply_circular_window(x, y, A, window_radius=0.05)
```

---

apply_lowpass_filter_2D function

```
apply_lowpass_filter_2D(x: ndarray, y: ndarray, data: ndarray, fmax: float) -> ndarray
```

Apply low-pass filter to 2D mirror map.

Parameters

`x, y`
: 1D coordinate axes.

`data`
: 2D map data to filter.

`fmax`
: Maximum frequency cutoff [1/units of x,y].

Returns

Low-pass filtered 2D map (real values).

Notes

Applies circular window in frequency domain with radius `fmax`, removing spatial frequencies above the cutoff.

4.5.5 Visualization

---

plot_psd1D function

```
plot_psd1D(f: ndarray, A: ndarray, figsize: tuple[float, float] = (6, 4.5),
    plottitle: str | None = None, funit: str = 'm$^{-1}$', ax: Axes | None = None,
    **kwargs) -> tuple[Figure, Axes]
```

Plot 1D FFT/PSD as linear spectral density.

Parameters

`f` Frequency axis.

`A` PSD data.

`figsize`
Figure size. Default (6, 4.5).

`plottitle`
Plot title. Default None.

`funit`
Frequency unit for x-axis label. Default "m^{-1}".

`ax`
Matplotlib axes to plot on. Default None creates new figure.

`**kwargs`
Additional arguments passed to `ax.loglog()`.

Returns

Tuple `(fig, ax)`: Matplotlib figure and axes objects.

Notes

Plots $\sqrt{\text{PSD}}$ as linear spectral density on log-log axes. Automatically creates legend if 'label' keyword is provided.

---

plot_psd2D function

```
plot_psd2D(fx: ndarray, fy: ndarray, A: ndarray, figsize: tuple[int, int] = (5, 5),
    Zmax: float | None = None, title: str | None = None, Z_label: str = '',
    log: bool = False, colormap: Any | None = None, ax: Axes | None = None,
    barshrink: float = 0.5, barpad: float = 0.1, showaxes: bool = True,
    debug: bool = False) -> tuple[Figure, Axes]
```

Plot 2D FFT/PSD data.

Parameters

`fx, fy`
2D frequency axes.

`A` 2D FFT/PSD data to plot.

`figsize`
Figure size. Default (5, 5).

`Zmax`
Color scale limits (float or tuple). Default None for auto-scaling.

`title`
Plot title. Default None.

`Z_label`
Colorbar label. Default "".

`log`
Use log scale for axes. Default False.

`colormap`
Matplotlib colormap. Default None uses `gwoptics_colormap`.

`ax`
Matplotlib axes to plot on. Default None creates new figure.

`barshrink`
Colorbar shrink factor. Default 0.5.

`barpad`
Colorbar padding. Default 0.1.

`showaxes`
Show axis labels. Default True.

`debug`
Print diagnostic information. Default False.

Returns

Tuple `(fig, ax)`: Matplotlib figure and axes objects.

Notes

Plots $\log \sqrt{A}$ with `pcolormesh`. If `log=True`, trims negative frequencies and uses log scale for both axes.

4.6 Optical coupling: `mirror_maps.optical`

Optical coupling analysis. Functions to analyse optical mode couplings from mirror surface maps using Finesse gravitational wave detector simulations. This module provides tools for:

- computing optical coupling coefficients for reflected beams;
- analyzing cavity circulating fields with surface maps applied;
- visualizing higher-order mode (HOM) power distributions.

Function	Purpose
optical_coupling_refl	Compute coupling on direct reflection from mirror
optical_coupling_cavity	Compute coupling in resonant cavity configuration
plot_HOM_histogram	Visualize HOM power distribution

Anna Green, December 2023

---

optical_coupling_refl

function

```
optical_coupling_refl(Mapxyz: tuple[ndarray, ndarray, ndarray] | None, maxtem: int = 5,
    detector: str = 'Virgo', optic: str = 'EM', aperture: bool = True,
    phase_setting: int = 2) -> tuple[Model, ndarray]
```

Compute optical coupling coefficients on reflection from a mirror.

Uses the mirror component directly on reflection, so the q parameter of the reflected beam matches the input.

Parameters

`Mapxyz`
: Tuple `(x, y, Z)` of coordinate arrays and surface height data, or None for ideal mirror.

`maxtem`
: Maximum transverse mode order. Default 5.

`detector`
: Detector configuration ("Virgo"). Default "Virgo".

`optic`
: Optic type ("EM"/"ETM"/"end" or "IM"/"ITM"/"input"). Default "EM".

`aperture`
: Whether to apply circular aperture from map extent. Default True.

`phase_setting`
: Phase configuration (2 or 3). Default 2.

Returns

Tuple `(model, E_refl)`: Finesse model and reflected field amplitudes.

Raises

`ValueError`
: If `detector` or `optic` is not supported.

Notes

Phase setting 2: `zero_k00=False`, `zero_tem00_gouy=True` (recommended). Phase setting 3: matches Finesse 3 pre-3.0a28 (physically incorrect).

To do

- Check if results differ significantly for beamsplitter case.
- Implement LIGO and ET detector configurations.

optical_coupling_cavity function

```
optical_coupling_cavity(Mapxyz: tuple[ndarray, ndarray, ndarray] | None, maxtem: int = 5,
    detector: str = 'Virgo', aperture: bool = True, locked: bool = True,
    lockwith: str = 'pd', phase_setting: int = 2, debug: bool = False)
    -> tuple[Model, ndarray]
```

Compute optical coupling in a resonant cavity with surface map.

Returns the circulating field as viewed directly on reflection from the optic with the map applied. Currently limited to applying the map to the end mirror in a Virgo cavity.

Parameters

`Mapxyz`
: Tuple `(x, y, Z)` of coordinate arrays and surface height data, or None for ideal mirror.

`maxtem`
: Maximum transverse mode order. Default 5.

`detector`
: Detector configuration ("Virgo"). Default "Virgo".

`aperture`
: Whether to apply circular aperture from map extent. Default True.

`locked`
: Whether to re-tune cavity to operating point. Default True.

`lockwith`
: Locking method — 'pd' (power) or 'ad' (amplitude). Default 'pd'.

`phase_setting`
: Phase configuration (2 or 3). Default 2.

`debug`
: Print tuning information. Default False.

Returns

Tuple `(model, E_refl)`: Finesse model and reflected field amplitudes.

Raises

`ValueError`
: If `detector` or `lockwith` is not supported.

Notes

Phase setting 2: `zero_k00=False`, `zero_tem00_gouy=True` (recommended). Phase setting 3: matches Finesse 3 pre-3.0a28 (physically incorrect).

plot_HOM_histogram function

```
plot_HOM_histogram(model: Model, fdout: ndarray, by_order: bool = False,
    show_orders: list[int] | None = None, logy: bool = True, ax: Axes | None = None,
    plottype: str = 'bar', colorcode: bool = False, cmap: Any = cm.Spectral_r,
    color: str = '0.4', grid: bool = False, **kwargs) -> Axes
```

Plot histogram of optical higher-order mode couplings.

Parameters

`model`
Finesse model containing mode information.

`fdout`
Field detector output array from `model.run()`.

`by_order`
If True, sum power per order $(n + m)$. Default False.

`show_orders`
List of orders to display. Default None (all orders).

`logy`
Use logarithmic y-scale. Default True.

`ax`
Matplotlib axes to plot on. Default None creates new figure.

`plottype`
Plot type — “bar” or “scatter”. Default “bar”.

`colorcode`
Color bars by mode order. Default False.

`cmap`
Matplotlib colormap for color coding. Default `cm.Spectral_r`.

`color`
Bar color when not color coding. Default ‘0.4’.

`grid`
Show grid lines. Default False.

`**kwargs`
Additional arguments passed to `plot_histogram()`.

Returns

The axes object (`matplotlib.axes.Axes`).

Raises

`TypeError`
If `show_orders` is provided but not a list.

Example

```
>>> model, E_refl = optical_coupling_refl(Mapxyz)
>>> ax = plot_HOM_histogram(model, E_refl, by_order=True, logy=True)
```

4.7 Colour map: `mirror_maps.seasons_s`

CHAPTER 5

Beam propagation and gratings

5.1 FFT propagation: `fft_propagation`

Collection of function for propagating a light beam using the FFT propagation method

Andreas Freise, 18.05.2025

Function	Purpose
`Grid`	Grid of a 2D array of complex beam amplitudes
`fft_propagate`	FFT propagation in a fixed grid
`fft_propagate_simple`	FFT propagation without the grid class
`fft_scale_propagate`	FFT propagation with an adaptive grid size
`apply_lens`	Phase factor of a lens
`apply_thin_lens`	Phase factor of a thin lens
`required_roundtrips`	Number of roundtrips for a cavity FFT simulation

5.1.1 The grid

`Grid` class

```
class Grid(_xpoints, _ypoints, _xlength, _ylength, _xoffset, _yoffset)
```

Bases: `object`

Data structure to describe the size and axes for a (x, y) data array of complex beam amplitudes. Also contains data structures for FFT propagation.

Notes

Some notes on usage (added by AJ 3.May.2018): when accessing an element of a `beam[i,j]`, `i` describes the x position and `j` describes the y position. When plotting using `imshow`, the positive x direction is down, the positive y direction is right.

5.1.2 Propagation

`fft_propagate` function

```
fft_propagate(field: ndarray, grid: Grid, wavelength: float, distance: float,
    refaction_index: float, bandlimit: bool = False) -> ndarray
```

Propagates a 2D field using the Fast Fourier Transform (FFT) in a fixed Grid.

Parameters

`field`
: [`np.ndarray`] The 2D field to propagate.

`grid`
: [`NdGrid`] The Grid on which the field is defined.

`wavelength`
: [`float`] The vacuum wavelength of light in meters.

`distance`
: [`float`] The distance over which to propagate the field in meters.

`refaction_index`
: [`int`] The index of refraction of the medium through which the field is propagating.

`bandlimit`
: [`bool, optional`] If True, apply the band-limited angular spectrum mask (Matsushima and Shimobaba, 2009 [1]) to the transfer function. This zeros spatial frequencies beyond the analytic cutoff at which the discrete kernel starts to alias, which otherwise produces artefacts in the propagated field. The mask is applied with the cutoff appropriate for the Fresnel paraxial kernel used here (see Notes). Has no effect when `distance == 0`. Default is False, preserving the original behaviour of this function.

Returns

`np.ndarray`
: The propagated 2D field.

Notes

The Fresnel transfer function used here is

$$H(f) = \exp\left(i\pi \frac{\lambda}{n} z \left(f_x^2 + f_y^2\right)\right).$$

Its local frequency along f_x is $(\lambda/n)\, z f_x$. Combined with the spectral grid step $df_x = 1/L_x$, the Nyquist criterion gives the cutoff

$$|f_x| \leq \frac{nL_x}{2\lambda z}$$

and analogously for f_y. Beyond this cutoff the discrete propagator samples its own oscillations at less than two samples per period, so its contribution wraps in the spatial domain. The `bandlimit` option masks this region to zero. This is the Fresnel-form variant of the band-limited angular spectrum method of [1]; the original derivation is for the full Rayleigh–Sommerfeld kernel and the cutoff is essentially the same in the paraxial limit.

`fft_propagate_simple` function

```
fft_propagate_simple(field: ndarray, xpoints: int, ypoints: int, xstep: int, ystep: int,
    lambda_: float, distance: float, nr: float) -> ndarray
```

FFT propagation code (not relying on `NdGrid` class).

Parameters

`field`
[`np.ndarray`] The initial field E.

`xpoints`
[`int`] Number of points in the Grid along the x-axis.

`ypoints`
[`int`] Number of points in the Grid along the y-axis.

`xstep`
[`float`] Physical size of one tile in the NdGrid along the x-axis.

`ystep`
[`float`] Physical size of one tile in the NdGrid along the y-axis.

`lambda_`
[`float`] Vacuum wavelength.

`distance`
[`float`] Distance over which to propagate in meters.

`nr`
[`float`] Index of refraction.

Returns

`field`
[`array_like`] The propagated field E.

`fft_scale_propagate` function

```
fft_scale_propagate(field: ndarray, grid1: Grid, grid2: Grid, wavelength: float,
    distance: float, scale: float, refraction_index: float) -> ndarray
```

Perform FFT propagation with an adaptive Grid size.

This function applies a Fourier transform to the input field, multiplies it by a phase factor that accounts for the propagation distance and scale, and then performs an inverse Fourier transform to obtain the output field. The scaling is performed to avoid aliasing errors in the FFT calculation.

Parameters

`field`
[`np.ndarray`] The input electric field as a 2D complex array.

`grid1`
[`NdGrid`] The initial Grid object that defines the spatial coordinates of the input field.

`grid2`
[NdGrid] The final Grid object that defines the spatial coordinates of the output field.

`wavelength`
[float] The vacuum wavelength of the light in meters.

`distance`
[float] The propagation distance in meters.

`scale`
[float] The scaling factor to apply to the input field before performing the FFT.

`refraction_index`
[float] The refractive index of the medium through which the light is propagating.

Returns

`np.ndarray`
The output electric field as a 2D complex array, with the same shape as the input field.

Notes

This function uses the scaling method described in [2] (pages 179–184). The scaling factor is given as A_1/A_0 with A_0 the Grid size at the start of propagation and A_1 the expected beam size at the end of propagation.

NOT YET TESTED

5.1.3 Lenses

`apply_lens` function

```
apply_lens(field: ndarray, grid: Grid, wavelength: float, focal_length: float) -> ndarray
```

Apply a phase factor to a 2D complex field amplitude representing the transmission of a lens.

Parameters

`field`
[np.ndarray] 2D numpy array representing a complex field amplitude before the lens.

`grid`
[Grid] The grid on which the field is defined.

`wavelength`
[float] Vacuum wavelength of light, in meters.

`focal_length`
[float] Focal length of the lens, in meters.

Returns

`field`
A 2D numpy array of the complex field amplitude after the lens.

`apply_thin_lens` function

```
apply_thin_lens(field: ndarray, grid: Grid, wavelength: float, focal_length: float) ->
        ndarray
```

Apply a phase factor representing a thin lens.

Parameters

`field`
: [np.ndarray] The initial field to be propagated.

`grid`
: [NdGrid] The Grid on which the field is defined.

`wavelength`
: [float] The vacuum wavelength of light, in meters.

`focal_length`
: [float] The focal length of the thin lens, in meters.

Returns

`np.ndarray`
: The propagated field after applying the thin lens phase factor.

5.1.4 Cavity simulations

`required_roundtrips` function

```
required_roundtrips(Loss, accuracy)
```

Estimates the required number of roundtrips for a cavity FFT simulation.

After infinitely many roundtrips the power is given by

$$P_{\text{total}} = \frac{P_{\text{in}}}{(1-r)^2},$$

after N roundtrips it is

$$P_N = P_{\text{in}} \frac{\left(1 - r^{N+1}\right)^2}{(1-r)^2},$$

with P_{in} the power transmitted into the cavity. So the relative error is

$$1 - \frac{P_N}{P_{\text{total}}} = 1 - \left(1 - r^{N+1}\right)^2,$$

which should be smaller than `accuracy`. This yields

$$\text{roundtrips} = \left\lceil \frac{\log\left(1 - \sqrt{1 - \text{accuracy}}\right)}{\log \sqrt{R}} \right\rceil - 1;$$

for accuracy $\ll 1$ we can simplify this.

5.2 Gratings: `grating`

Simple functions related to modelling gratings
Andreas Freise, 18.05.2025

`grating_sin` function

```
grating_sin(amplitude: float, period: float, mirror_radius: float, npoints: int,
    offset: float = 0.0) -> tuple[ndarray, ndarray, ndarray]
```

Generate a sinusoidal grating along x on a circular disk.

Parameters

`period`
: [`float`] Period of the grating [m].

`mirror_radius`
: [`float`] Radius of the disk on which the grating is placed [m].

`npoints`
: [`int`] Number of points for the disk along x and y.

`offset`
: [`float`] Fraction of a period to offset the grating along x.

Returns

`x` [`ndarray`] 1D array representing the axis along x.

`y` [`ndarray`] 1D array representing the axis along y.

`surface`
: [`ndarray`] 2D array with the surface height of the grating [m].

`grating_steps` function

```
grating_steps(amplitude: float, period: float, mirror_radius: float, npoints: int,
    offset: float = 0) -> tuple[ndarray, ndarray, ndarray]
```

Generate a grating with a rectangular structure along x on a circular disk.

Parameters

`period`
: [`float`] Period of the grating [m].

`mirror_radius`
: [`float`] Radius of the disk on which the grating is placed [m].

`npoints`
: [`int`] Number of points for the disk along x and y.

`offset`
: [`float, optional`] Fraction of a period to offset the grating along x (default is 0).

`duty_cycle`
: [float, optional] Fraction of cycle during which the rectangular structure is at high amplitude (default is 0.5).

Returns

`x` [ndarray] 1D array representing the axis along x.

`y` [ndarray] 1D array representing the axis along y.

`surface`
: [ndarray] 2D array with the surface height of the grating [m].

References

Bibliography

[1] K. Matsushima and T. Shimobaba, "Band-Limited Angular Spectrum Method for Numerical Simulation of Free-Space Propagation in Far and Near Fields", Optics Express **17**, 19662 (2009). https://doi.org/10.1364/OE.17.019662

[2] J.-Y. Vinet, *The Virgo Physics Book: OPTICS and related TOPICS*. https://www.oca.eu/images/Artemis/pdf_artemis/Vinet-Optics_and_related_topics-VPB_2020.pdf

CHAPTER 6

General utilities

6.1 Top-level functions: `ifotools`

`print_expression_eval` function

```
ifotools.print_expression_eval(expression: Expr, comment: None | str = None,
    **kwargs: dict) -> None
```

Pretty prints a SymPy symbolic expression, equating it to its numerical value.

Parameters

`expression`
: [`sympy.Expr`] Expression to pretty print.

`comment`
: [`None | str, optional`] Comment to print above the expression, by default None.

`kwargs`
: [`dict`] Keyword arguments to pass to `sympy.pretty_print`. See `https://docs.sympy.org/latest/modules/printing.html#sympy.printing.pretty.pretty.pretty_print`

Example

```
>>> expr = sympy.Matrix([sympy.sqrt(2), 0])
>>> print_expression_eval(expr, "Pretty printed vector:")
Pretty printed vector:
```

$$\begin{bmatrix}\sqrt{2}\\0\end{bmatrix} = \begin{bmatrix}1.4142135623731\\0\end{bmatrix}$$

`search` function

```
ifotools.search(pattern: str | None = None, show_docstrings: bool = True,
    matching_lines: bool = True, color: bool = True)
```

6.2 Searching the package: `helper`

Helper function to search through the package function name and docstring
Miron van der Kolk, 21.05.2025

---

search

function

```
search(pattern: str | None = None, show_docstrings: bool = True,
    matching_lines: bool = True, color: bool = True)
```

Package

class

```
class Package(modules: dict[str, Module])
```

Bases: `object`

`modules: dict[str, Module]`

`search(pattern: str, show_docstrings: bool, matching_lines: bool, color: bool = True)`

Module

class

```
class Module(functions: dict[str, Function])
```

Bases: `object`

`functions: dict[str, Function]`

`search(pattern: str) -> tuple[Function]`

Function

class

```
class Function(signature: str, docstring: str | None = None)
```

Bases: `object`

`signature: str`

`docstring: str | None = None`

`match(pattern: str) -> bool`

Whether pattern exist in the function. Does case-insensitive literal match.

`pattern`

[`str`] Pattern to search for.

`returns`

[`bool`] Whether match is found.

`pretty_print(highlight: str = '', show_docstrings: bool = True, matching_lines: bool = False, color: bool = True)`

Pretty prints the function using the pygments terminal formatter. Highlights the `highlight` string in underlined red.

`highlight`
: [`str`] Substring to highlight.

`show_docstrings`
: [`bool`] Whether to include the function docstring.

`matching_lines`
: [`bool`] Whether to only print lines containing `highlight`.

`color`
: [`bool`] Whether to syntax highlight the results.

`get_code_before` function

```
get_code_before(line: str, sub: str) -> str
```

Get latest ANSI code before substring.

Parameters

`line`
: [`str`] Line to check.

`sub`
: [`str`] Substring before which to search.

Returns

`str`
: Latest ANSI code before `sub`. Returns empty string if no ANSI code is found.

`lower_case_match` function

```
lower_case_match(pattern: str, string: str) -> str
```

6.3 SymPy helpers: `sympy_helpers`

`print_expression_eval` function

```
print_expression_eval(expression: Expr, comment: None | str = None, **kwargs: dict) -> None
```

Pretty prints a SymPy symbolic expression, equating it to its numerical value.

Parameters

`expression`
: [`sympy.Expr`] Expression to pretty print.

`comment`
: [None | str, optional] Comment to print above the expression, by default None.

`kwargs`
: [dict] Keyword arguments to pass to `sympy.pretty_print`. See `https://docs.sympy.org/latest/modules/printing.html#sympy.printing.pretty.pretty.pretty_print`

Example

```
>>> expr = sympy.Matrix([sympy.sqrt(2), 0])
>>> print_expression_eval(expr, "Pretty printed vector:")
Pretty printed vector:
```

$$\begin{bmatrix}\sqrt{2}\\0\end{bmatrix} = \begin{bmatrix}1.4142135623731\\0\end{bmatrix}$$

6.4 General utilities: `utils`

Function	Purpose
`proportionality_factor`	Proportionality factor of a variable to a parameter
`partial_derivative`	Symbolic partial derivative of a Finesse function
`expand_function_with_spaces`	Expanded symbol with spaces around operators
`cart2pol`	Cartesian to polar coordinates
`color_text, pad`	Text formatting
`ANSICode`	ANSI escape codes

`proportionality_factor` function

```
proportionality_factor(variable: Parameter, parameter: Parameter) -> int | float
```

Compute the proportionality factor of a Finesse variable with respect to a parameter. The expression must be a linear function of the given parameter. If the expression does not depend on the parameter, this function returns 0.

The symbolic derivative of the variable expression with respect to the given parameter is computed, after which all other parameter values are substituted. The result is the proportionality factor.

Parameters

`expr`
: [Parameter] The Finesse variable.

`parameter`
: [Parameter] Parameter to differentiate with respect to.

Returns

`proportionality_factor`
: Constant of proportionality between variable expression and parameter.

`partial_derivative` function

```
partial_derivative(expr: Function, parameter: Parameter) -> int | float | Function
```

Compute the partial derivative of a symbolic Finesse function with respect to a Finesse parameter.

The expression is converted into a SymPy expression which is differentiated with respect to the parameter. If symbolic, the derivative is converted back into a Finesse function.

Parameters

`expr`
[`Parameter`] The Finesse variable.

`parameter`
[`Parameter`] Parameter to differentiate with respect to.

Returns

`proportionality_factor`
Constant of proportionality between variable expression and parameter.

`expand_function_with_spaces` function

```
expand_function_with_spaces(sym: Symbol, format: str | None = None) -> str
```

Expands a symbol and puts spaces around the operators '`+-/*`'.

Parameters

`sym`
[`Symbol`] Symbol to expand.

Returns

`str`
String version of the symbol with spaces around operators.

`cart2pol` function

```
cart2pol(x, y) -> tuple[float, float]
```

Converts from Cartesian to polar coordinates.

`color_text` function

```
color_text(text: str, color: ANSICode)
```

---

pad function

```
pad(text, width)
```

ANSICode class

```
class ANSICode(*values)
```

Bases: StrEnum

BOLD = '\x1b[1m'

GREEN = '\x1b[92m'

RED = '\x1b[91m'

RESET = '\x1b[0m'

WHITE = '\x1b[97m'

YELLOW = '\x1b[93m'

6.5 Version: version

Index of functions and classes

adjust_cavity_length (*finesse.pretuning*), 15
adjust_path_length (*finesse.pretuning*), 15
adjust_prc_length (*virgo.pretuning*), 30
adjust_src_length (*virgo.pretuning*), 31
adjust_variable_length (*finesse.pretuning*), 16
Anm (*mirror_maps.zernike*), 72
ANSICode (*utils*), 118
append_history (*mirror_maps.container*), 42
apply_circular_mask (*mirror_maps.utils*), 46
apply_circular_window (*mirror_maps.psd*), 99
apply_lens (*fft_propagation*), 108
apply_lowpass_filter_2D (*mirror_maps.psd*), 99
apply_ptc_finesse (*mirror_maps.utils*), 54
apply_thin_lens (*fft_propagation*), 109

BaseStatus (*finesse.status*), 19

calculate_recycling_cavity_length (*virgo.pretuning*), 29
cart2pol (*utils*), 117
check_map_NaNs (*mirror_maps.utils*), 50
color (*mirror_maps.utils*), 65
color_text (*utils*), 117
compare_models (*finesse.model_info*), 3
compute_annulus_rms (*mirror_maps.utils*), 57
coupling_coefficients (*mirror_maps.hermite_gauss*), 89
create_amplitude_from_nans (*mirror_maps.utils*), 51
create_file (*mirror_maps.container*), 37
create_polar_grid (*mirror_maps.utils*), 64
create_ptc_map (*mirror_maps.utils*), 55
create_random_map (*mirror_maps.utils*), 63
create_random_rect_map (*mirror_maps.utils*), 63
crop_nans (*mirror_maps.utils*), 47
crop_to_circle (*mirror_maps.utils*), 47
crop_to_rect (*mirror_maps.utils*), 48

display_dict (*mirror_maps.utils*), 64

entries (*mirror_maps.container*), 34
etalon_status (*finesse.etalon*), 17
EtalonStatus (*finesse.status*), 19
expand_function_with_spaces (*utils*), 117
extend_grid (*mirror_maps.utils*), 48

fft_propagate (*fft_propagation*), 106
fft_propagate_simple (*fft_propagation*), 107
fft_scale_propagate (*fft_propagation*), 107
fill_circle (*mirror_maps.utils*), 66
fill_NaN (*mirror_maps.utils*), 50
fill_NaNs_mean (*mirror_maps.utils*), 65
fill_NaNs_ndimage (*mirror_maps.utils*), 51
find_mirror_pairs (*finesse.etalon*), 18
fit_zernike (*mirror_maps.zernike*), 78
Function (*helper*), 114

generate_imports (*virgo.pretuning_generator*), 32
generate_pretuning_code (*virgo.pretuning_generator*), 32
get_Bn (*mirror_maps.zernike*), 81
get_cnm (*mirror_maps.zernike*), 76
get_cnm_robust (*mirror_maps.zernike*), 77
get_code_before (*helper*), 115
get_etalon_info (*finesse.etalon*), 18
get_hg_coeffs (*mirror_maps.hermite_gauss*), 87
get_hg_coeffs_wls (*mirror_maps.hermite_gauss*), 88
get_i (*mirror_maps.zernike*), 72
get_j_Noll (*mirror_maps.zernike*), 73
get_lock_pairs (*finesse.utils*), 22
get_model_elements (*finesse.utils*), 22
get_model_state (*virgo.model_info*), 25
get_nm (*mirror_maps.zernike*), 73

get_nm_Noll (*mirror_maps.zernike*), 74
get_phase_config_status (*finesse.model_info*), 5
get_powers (*finesse.model_info*), 5
get_powers (*virgo.model_info*), 26
get_ptc_finesse (*mirror_maps.utils*), 52
get_qnls (*virgo.model_info*), 27
get_status (*finesse.status*), 21
get_Znm_name (*mirror_maps.zernike*), 75
grating_sin (*grating*), 110
grating_steps (*grating*), 110
Grid (*fft_propagation*), 105
gwoptics_colormap (*mirror_maps.utils*), 62

hg_basis (*mirror_maps.hermite_gauss*), 86
hg_mode_indices (*mirror_maps.hermite_gauss*), 87

info (*mirror_maps.container*), 34
interpolate_NaNs (*mirror_maps.utils*), 66
is_entry (*mirror_maps.container*), 35
is_odd (*mirror_maps.utils*), 64

list_nm (*mirror_maps.zernike*), 74
load_finesse2 (*mirror_maps.utils*), 45
load_h5 (*mirror_maps.utils*), 43
load_metropro (*mirror_maps.utils*), 44
load_model (*finesse.utils*), 21
load_zeiss (*mirror_maps.utils*), 44
lower_case_match (*helper*), 115

make_hg_map (*mirror_maps.hermite_gauss*), 89
make_pyramid_figure (*mirror_maps.zernike*), 84
make_zernike_map (*mirror_maps.zernike*), 79
map_coverage (*mirror_maps.utils*), 52
map_rms (*mirror_maps.utils*), 56
map_to_psd1D (*mirror_maps.psd*), 94
max_radius (*mirror_maps.utils*), 49
ModelStateItem (*finesse.status*), 21
Module (*helper*), 114

Noll_order_basis (*mirror_maps.zernike*), 75

optical_coupling_cavity (*mirror_maps.optical*), 103
optical_coupling_refl (*mirror_maps.optical*), 102
optimise_demod_phases (*finesse.pretuning*), 16

Package (*helper*), 114
pad (*utils*), 118
partial_derivative (*utils*), 117
PhaseConfigStatus (*finesse.status*), 20
plot_basis (*mirror_maps.zernike*), 84
plot_Bn_histogram (*mirror_maps.zernike*), 85
plot_cnm_histogram (*mirror_maps.zernike*), 85
plot_compass (*finesse.plotting*), 12
plot_data_2D (*mirror_maps.utils*), 61
plot_dof_scan (*finesse.plotting*), 9
plot_dof_scans (*finesse.plotting*), 10
plot_error_signals (*finesse.plotting*), 11
plot_histogram (*mirror_maps.utils*), 61
plot_HOM_histogram (*mirror_maps.optical*), 103
plot_map (*mirror_maps.utils*), 58
plot_map_crossection (*mirror_maps.utils*), 59
plot_psd1D (*mirror_maps.psd*), 100
plot_psd2D (*mirror_maps.psd*), 100
plot_qnls (*virgo.plotting*), 28
plot_radial_profile (*mirror_maps.utils*), 60
plot_sensing_matrix (*finesse.plotting*), 13
plot_zernike_pyramid (*mirror_maps.zernike*), 83
plot_Znm (*mirror_maps.zernike*), 83
prepare_map (*mirror_maps.utils*), 54
pretune (*virgo.pretuning*), 31
print_dc_readout_powers (*finesse.model_info*), 6
print_dofs (*finesse.model_info*), 6
print_expression_eval (*ifotools*), 113
print_expression_eval (*sympy_helpers*), 115
print_lengths_frequencies (*virgo.model_info*), 28
print_lock_status (*finesse.model_info*), 7
print_model_state (*virgo.model_info*), 25
print_powers (*finesse.model_info*), 6
print_powers (*virgo.model_info*), 26
print_sensing_matrix (*finesse.model_info*), 7
print_sidebands (*finesse.model_info*), 8
print_tunings (*finesse.model_info*), 8
proportionality_factor (*utils*), 116
psd (*mirror_maps.psd*), 92
psd1D_rms (*mirror_maps.psd*), 96
psd1D_to_2D (*mirror_maps.psd*), 98

psd1D_to_map (*mirror_maps.psd*), 97
psd2D (*mirror_maps.psd*), 93
psd2D_cross_section (*mirror_maps.psd*), 95
psd2D_rms (*mirror_maps.psd*), 96

radial_average_map (*mirror_maps.utils*), 57
radial_average_psd (*mirror_maps.psd*), 94
random_cnm_Bn (*mirror_maps.zernike*), 82
random_cnm_flat (*mirror_maps.zernike*), 82
random_cnm_simtools (*mirror_maps.zernike*), 82
read_attrs (*mirror_maps.container*), 36
read_map (*mirror_maps.container*), 35
read_samples (*mirror_maps.container*), 36
read_value (*mirror_maps.container*), 36
recentre_map (*mirror_maps.utils*), 49
remove_curvature (*mirror_maps.utils*), 67
remove_entry (*mirror_maps.container*), 41
remove_mean (*mirror_maps.utils*), 55
remove_NaN (*mirror_maps.utils*), 51
remove_ptc_finesse (*mirror_maps.utils*), 53
remove_ptc_zernike (*mirror_maps.zernike*), 80
remove_tilt_piston (*mirror_maps.utils*), 67
required_roundtrips (*fft_propagation*), 109
resolve_dof_scan_pair_detector (*finesse.utils*), 23
resolve_dof_scan_pairs (*finesse.utils*), 23
resolve_error_signal_scan_pairs (*finesse.utils*), 24
rms_from_cnm (*mirror_maps.zernike*), 79
Rnm (*mirror_maps.zernike*), 71
Rnm_recurrence (*mirror_maps.zernike*), 71

save_h5 (*mirror_maps.utils*), 43
save_plot_pdf (*mirror_maps.utils*), 45
scattering_loss (*mirror_maps.hermite_gauss*), 90
search (*helper*), 114
search (*ifotools*), 113
set_cbar_extent (*mirror_maps.utils*), 62
set_recommended (*mirror_maps.container*), 40
StandardStatus (*finesse.status*), 20
summary (*finesse.model_info*), 4

test_all_mirrors (*finesse.etalon*), 18
tukey_window (*mirror_maps.psd*), 98

validate (*mirror_maps.container*), 37

write_attrs (*mirror_maps.container*), 39
write_map (*mirror_maps.container*), 38
write_value (*mirror_maps.container*), 39

zernike_basis (*mirror_maps.zernike*), 75
zero_cnm_terms (*mirror_maps.zernike*), 81
Znm (*mirror_maps.zernike*), 70